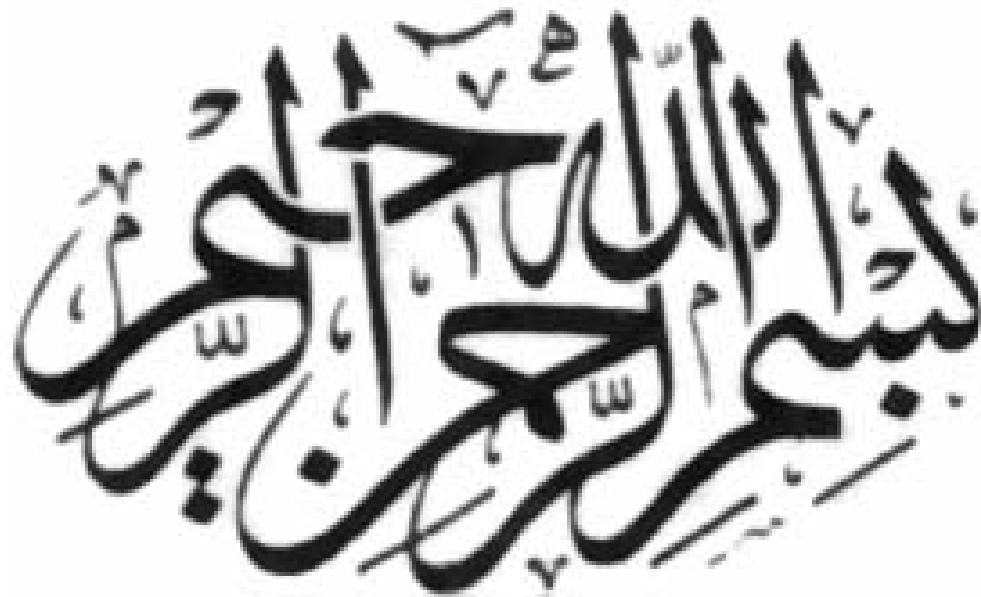




Basic building blocks design options

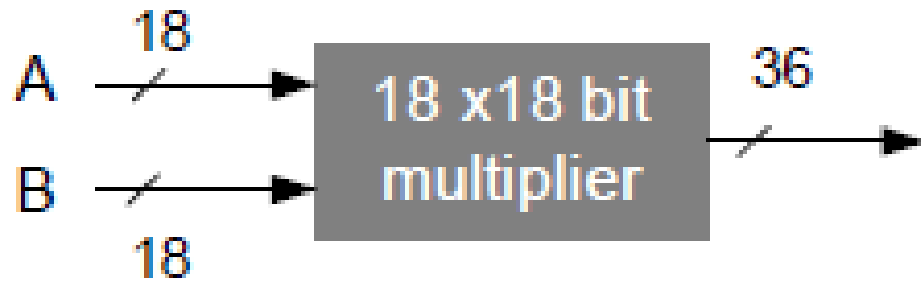
Introduction

- FPGAs outperform their traditional competing technology of digital signal processors (DSPs).
- No matter how many MACs the DSP vendor can place on a chip, still it cannot compete with the availability of hundreds of these units on a high end FPGA device.
- The modern day FPGAs come with embedded processors, standard interfaces and signal processing building blocks consisting of multipliers, adders, registers and multiplexers.
- It is expected that the number and type of these building blocks on FPGAs will see an upward trend

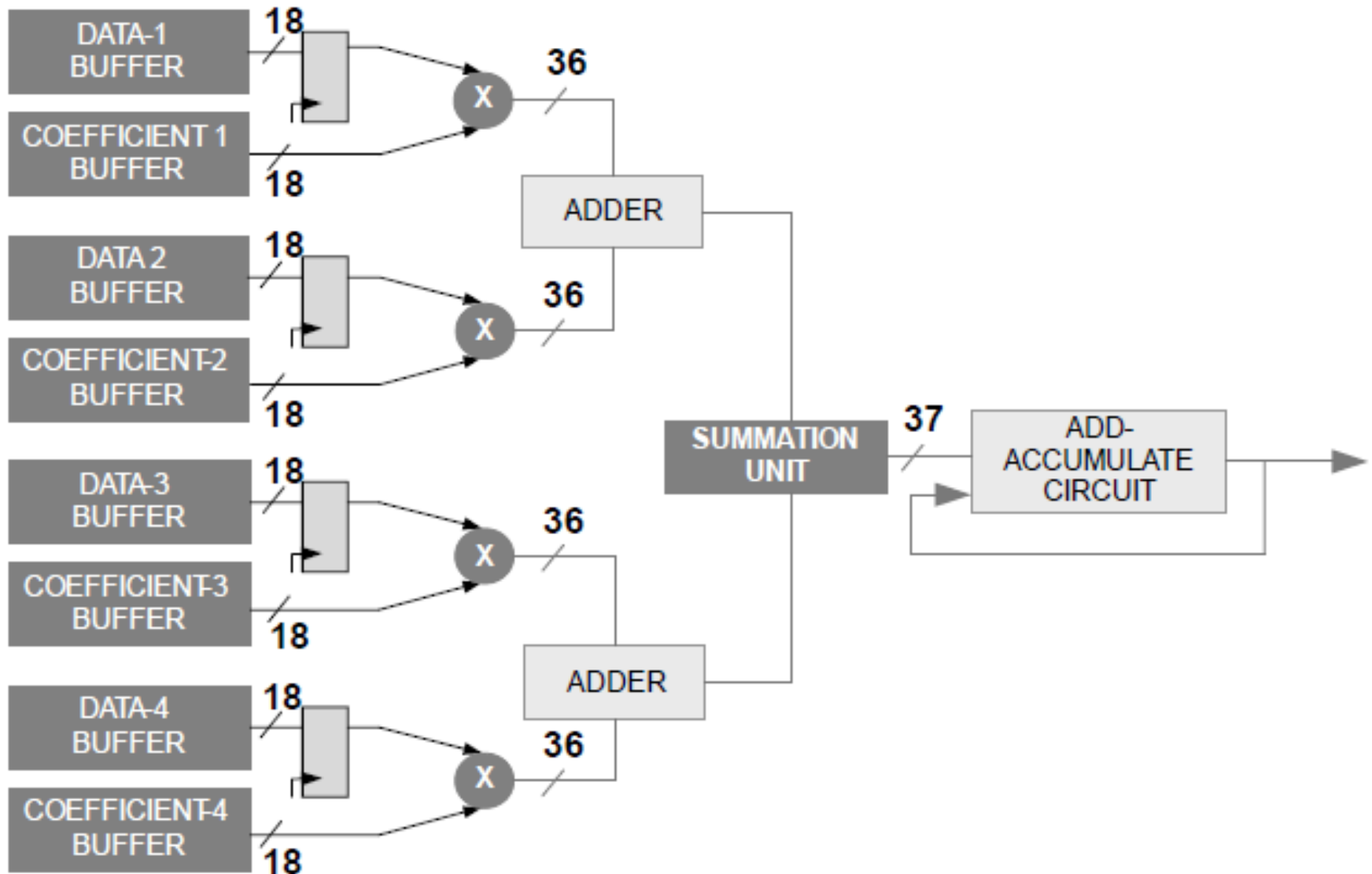
Dedicated computational blocks

- 18x18 multiplier in Virtex II, Virtex II pro and Spartan 3 FPGA
- 8x8 multiplier and 16 bit adder in Quick Logic FPGA
- 18x18 multiplier and adder in Altera FPGA
- DSP48 blocks in Xilinx 7 series FPGAs

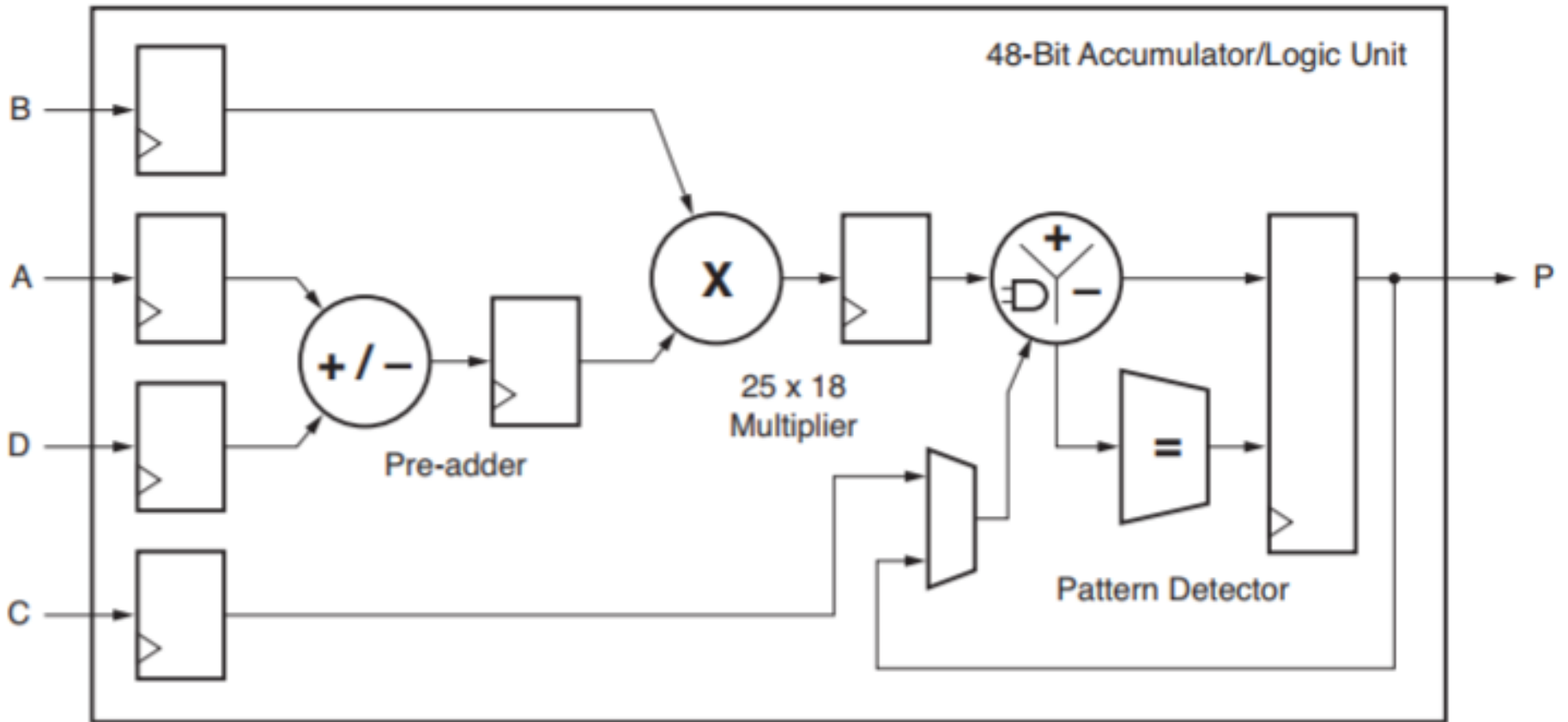
18x18 multiplier



18x18 multiplier and adder in Altera



DSP48 blocks in Xilinx



Embedded processors

- The FPGA vendors are also incorporating cores of programmable processors and numerous high speed interfaces.
- High end devices in Xilinx FPGAs are embedded with Hard IP core of PowerPC, ARM cortex or Soft IP core of Microblaze, along with standard interfaces like PCI Express and Gigabit Ethernet

FPGA embedded processors and other interfaces

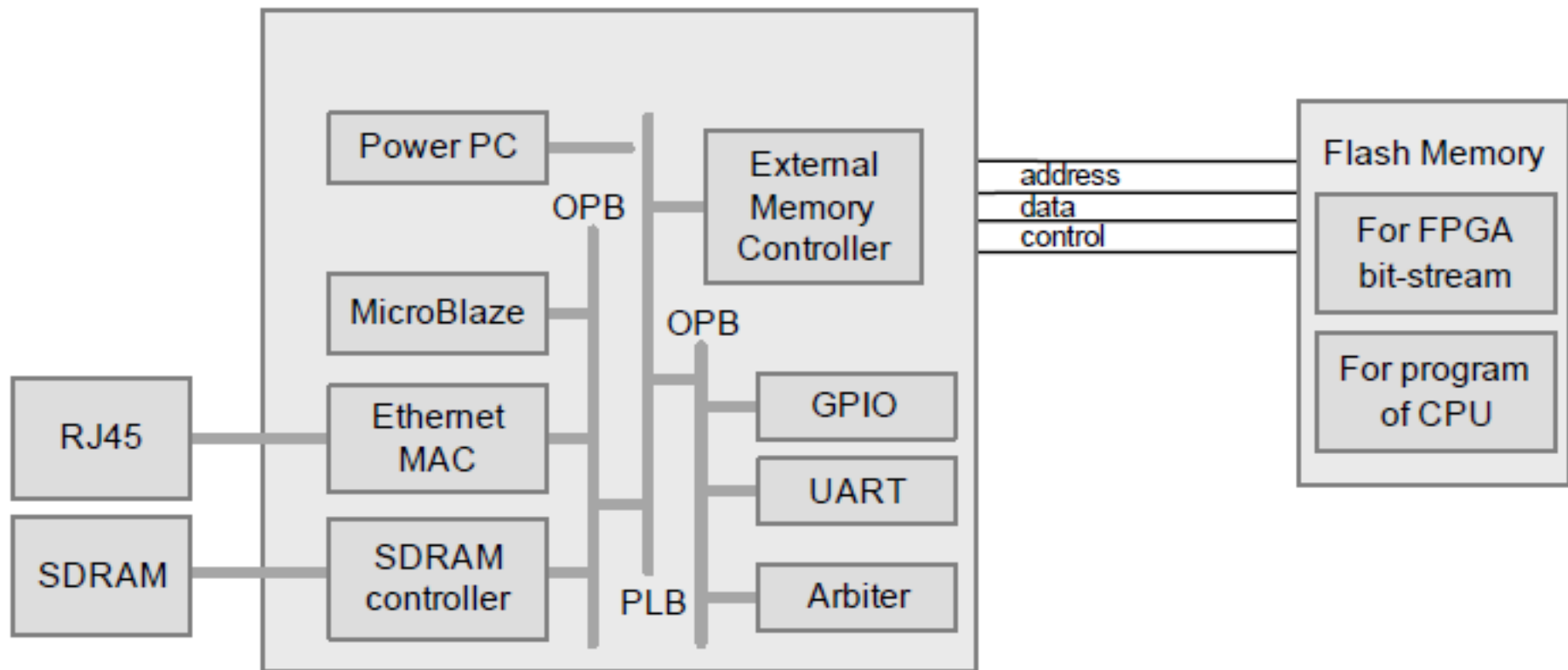


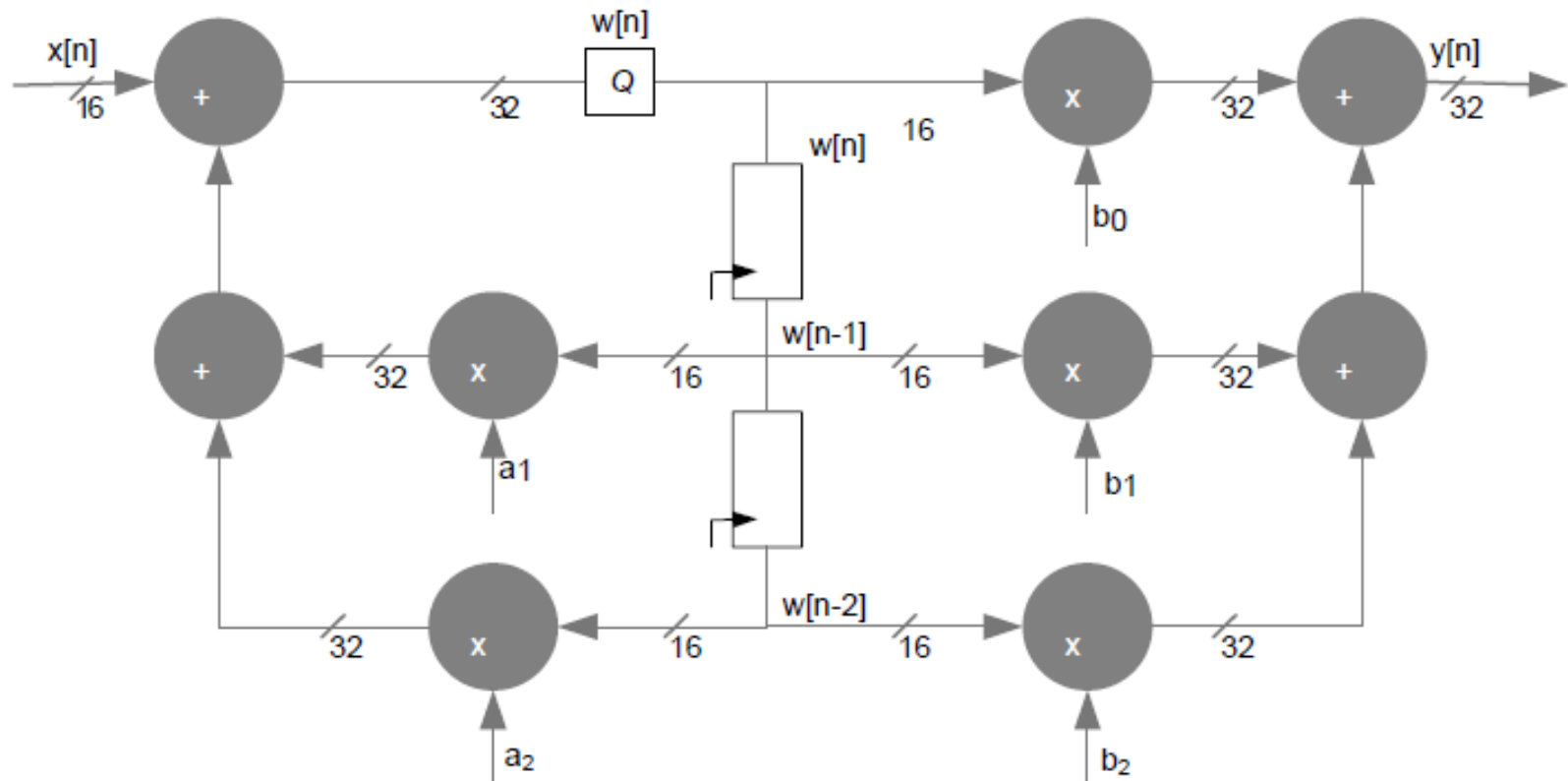
Figure 5.2 FPGA with PowerPC, MicroBlaze, Ethernet MAC and other embedded interfaces

Instantiation of Embedded Blocks

- Example: a second order infinite impulse response (IIR) filter in Direct Form (DF) II realization

$$w[n] = a_1 w[n-1] + a_2 w[n-2] + x[n]$$

$$y[n] = b_0 w[n] + b_1 w[n-1] + b_2 w[n-2].$$

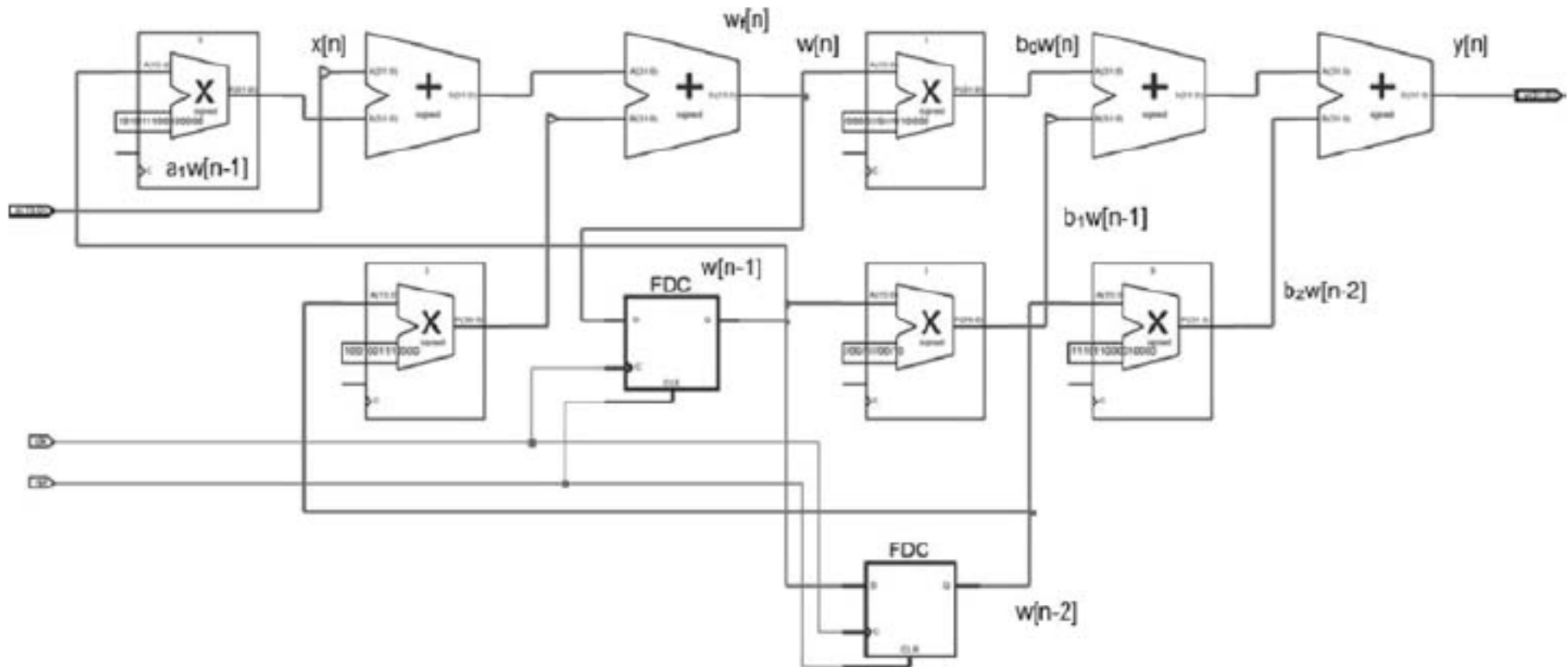


Code

```
always @(posedge clk or posedge rst)
begin
    if(rst)
        begin
            wn_1 < #1 0;
            wn_2 < #1 0;
            yn < #1 0;
        end
    else
        begin
            wn_1 < #1 wn;
            wn_2 < #1 wn_1;
            yn < #1 b0*wn + b1*wn_1 + b2*wn_2; // computing y[n] in Q2.30
                                                format with one redundant
                                                sign bit
        end
    end
end
endmodule
```

Spartan-3 architecture

- Multiplication is implemented using 18x18 dedicated multipliers



Device utilization summary

Spartan-3

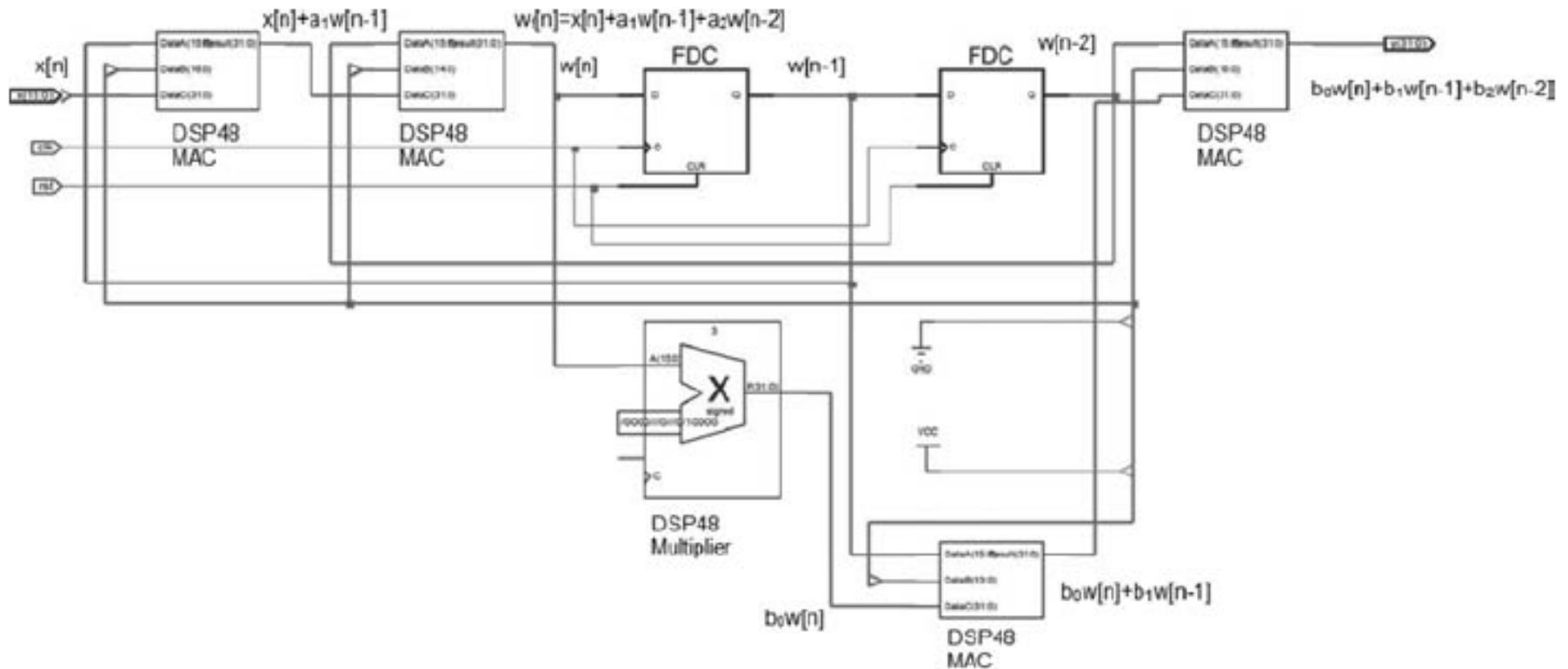
Selected device: 3s400pq208-5

Minimum period: 10.917 ns (maximum frequency: 91.597 MHz)

Number of slices:	58 out of 3584	1%
Number of slice flip-flops:	32 out of 7168	0%
Number of 4-input LUTs:	109 out of 7168	1%
Number of IOs:	50	
Number of bonded IOBs:	50 out of 141	35%
Number of multi 18 × 18s:	5 out of 16	31%
Number of GCLKs:	1 out of 8	12%

Virtex-4 architecture

- The multiplication and addition operations are mapped on DSP48 multiply accumulate (MAC) embedded blocks.



Device utilization summary

Virtex-4

Selected device: 4vlx15sf363-12

Minimum period: 7.664 ns (maximum frequency: 130.484 MHz)

Number of slices:	17 out of 6144	0%
Number of slice flip-flops:	32 out of 12288	0%
Number of 4-input LUTs:	16 out of 12288	0%
Number of IOs:	50	
Number of bonded IOBs:	50 out of 240	20%
Number of GCLKs:	1 out of 32	3%
Number of DSP48s:	5 out of 32	15%

Design optimization by pipe-lining

- Example-2: 8 tap FIR filter

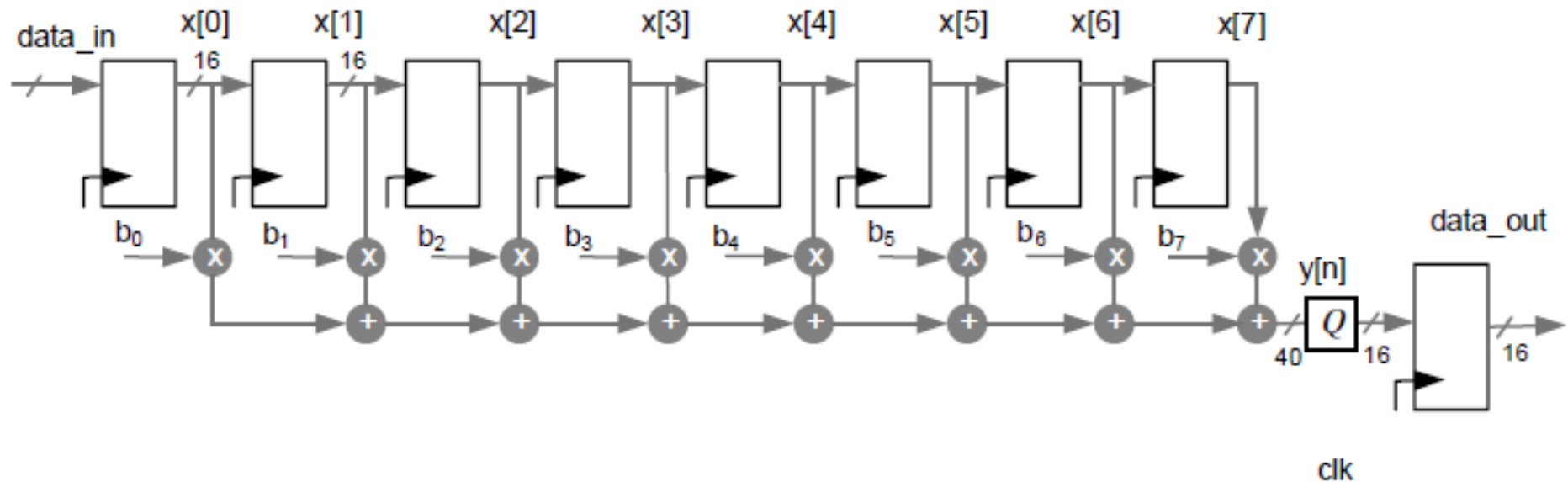


Figure 5.4 An 8 tap direct form (DF I) FIR filter

RTL code: FIR

```
// Module: fir filter
// Discrete-Time FIR Filter
// Filter Structure : Direct-Form FIR
// Filter Order : 7
// Input Format: Q1.15
// Output Format: Q1.15
module fir_filter (
    input clk,
    input signed [15:0] data_in, //Q1.15
    output reg signed [15:0] data_out //Q1.15
);
// Constants, filter is designed using Matlab Fdatool, all coefficients are in
// Q1.15 format
parameter signed [15:0] b0    16'b1101110110111011;
parameter signed [15:0] b1    16'b1110101010001110;
parameter signed [15:0] b2    16'b0011001111011011;

parameter signed [15:0] b3    16'b01101000000001000;
parameter signed [15:0] b4    16'b01101000000001000;
parameter signed [15:0] b5    16'b0011001111011011;
parameter signed [15:0] b6    16'b1110101010001110;
parameter signed [15:0] b7    16'b1101110110111011;
```

RTL code: FIR

```
reg signed [15:0] xn [0:7]; // input sample delay line
wire signed [39:0] yn; // Q8.32
// Block Statements
always @(posedge clk)
begin
    xn[0] < data in;
    xn[1] < xn[0];
    xn[2] < xn[1];
    xn[3] < xn[2];
    xn[4] < xn[3];
    xn[5] < xn[4];
    xn[6] < xn[5];
    xn[7] < xn[6];
    data out < yn[30:15]; // bring the output back in Q1.15
                        //format
end
assign yn xn[0] * b0 + xn[1] * b1 + xn[2] * b2 + xn[3] * b3 + xn[4] * b4 + xn[5]
    * b5 + xn[6] * b6 + xn[7] * b7;
endmodule
```

Schematic Spartan-3

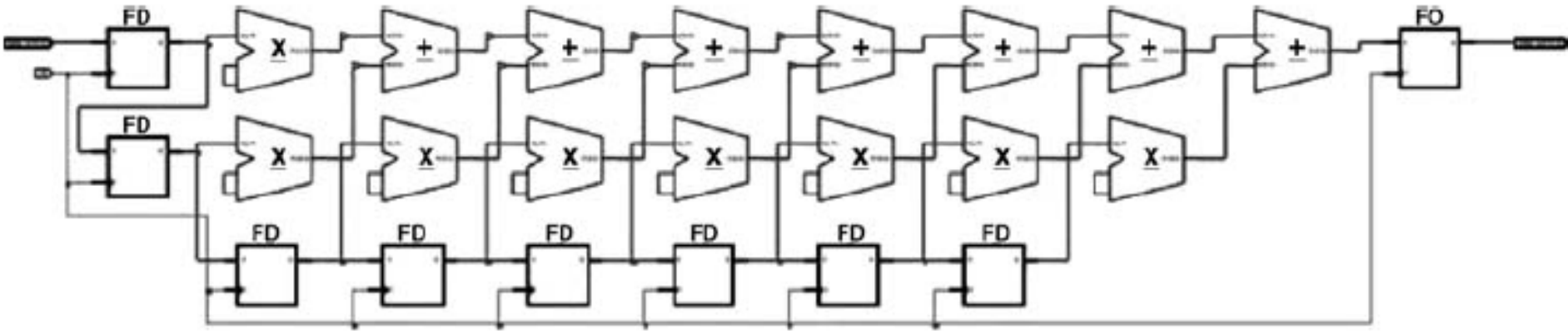


Figure 5.5 Schematic displaying the use of eight 18×18 embedded multiplier of Spartan™ 3 FPGA

Device utilization

Table 5.1 Synthesis reports: (a) Eight 18×18 bit embedded multipliers and seven adders from generic logic blocks are used on a Spartan™ 3 family of FPGA. (b) Eight DSP48 embedded blocks are used once mapped on a Vertex™ 4 family of FPGA

(a)			(b)		
Selected device: 3s200pq208 5			Selected device: 4vlx15sf363 12		
Minimum period: 23.290 ns			Minimum period: 16.958 ns		
(Maximum frequency: 42.936 MHz)			(Maximum frequency: 58.969 MHz)		
Number of slices:	185 out of 1920	9%	Number of Slices:	9 out of 6144	0%
Number of Slice	144 out of 3840	3%	Number of Slice	16 out of 12288	0%
Flip Flops:			Flip Flops:		
Number of 4 input	217 out of 3840	5%	Number of	33	
LUTs:			IOs:		
Number of	33		Number of bonded	33 out of 240	13%
IOs:			IOBs:		
Number of bonded	33 out of 141	23%	Number of GCLKs:	1 out of 32	3%
IOBs:					
Number of	8 out of 12	66%	Number of DSP48s:	8 out of 32	25%
MULT 18×18 s:					
Number of GCLKs:	1 out of 8	12%			

Design optimization

- Introducing pipeline registers

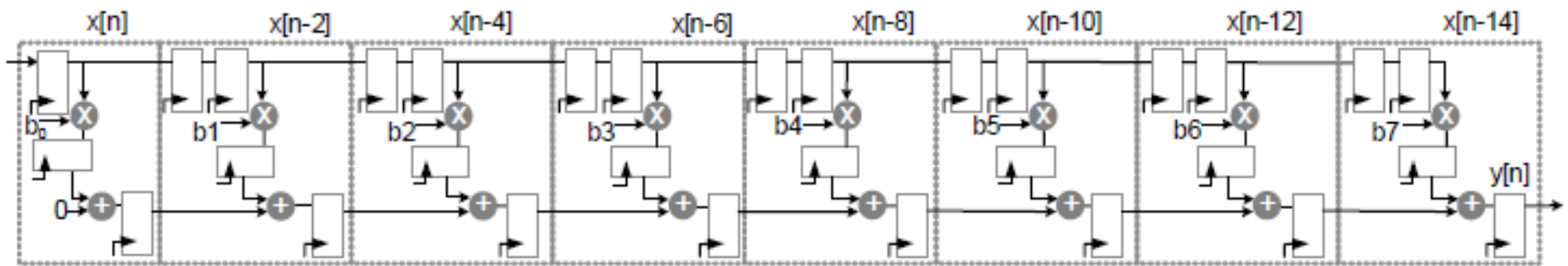


Figure 5.6 Pipeline implementation of 8 tap FIR filter optimized for mapping on FPGAs with DSP48 embedded blocks

RTL code

```
integer i;
always @(posedge clk)
    begin
        xn[0] < data in;
        for (i 0; i<14; i i+1)
            xn[i+1] xn[i];
        data out < yn[30:15]; // bring the output back in Q1.15 format
    end
always @(posedge clk)
begin
    prod[0] < xn[0] * b0;
    prod[1] < xn[2] * b1;
    prod[2] < xn[4] * b2;
    prod[3] < xn[6] * b3;
    prod[4] < xn[8] * b4;
    prod[5] < xn[10] * b5;
    prod[6] < xn[12] * b6;
    prod[7] < xn[14] * b7;
end
always @(posedge clk)
begin
    mac[0] < prod[0];
    for (i 0; i<7; i i+1)
        mac[i+1] < mac[i]+prod[i+1];
    end
assign yn mac[7];
```

Synthesis report

- Design is 9 times faster

Table 5.2 Synthesis report of mapping pipelined 8 tap FIR filter on Virtex™ 4 family of FPGA

Selected device: 4vlx15sf363 12		
Minimum period: 1.891 ns		
(Maximum frequency: 528.821 MHz)		
Number of slices:	9 out of 6144	0%
Number of slice flip flops:	16 out of 12288	0%
Number of I/Os:	33	
Number of bonded IOBs:	33 out of 240	13%
Number of GCLKs:	1 out of 32	3%
Number of DSP48s:	8 out of 32	25%

Basic Building Blocks architecture

- After the foregoing discussion of the use of dedicated multipliers and MAC blocks, it is pertinent to look at the architectures for the basic building blocks
- Several architectural options are available for selecting an appropriate HW block for operations like addition, multiplication and shifting
 - Parallel adders
 - Barrel shifters
 - Parallel multipliers

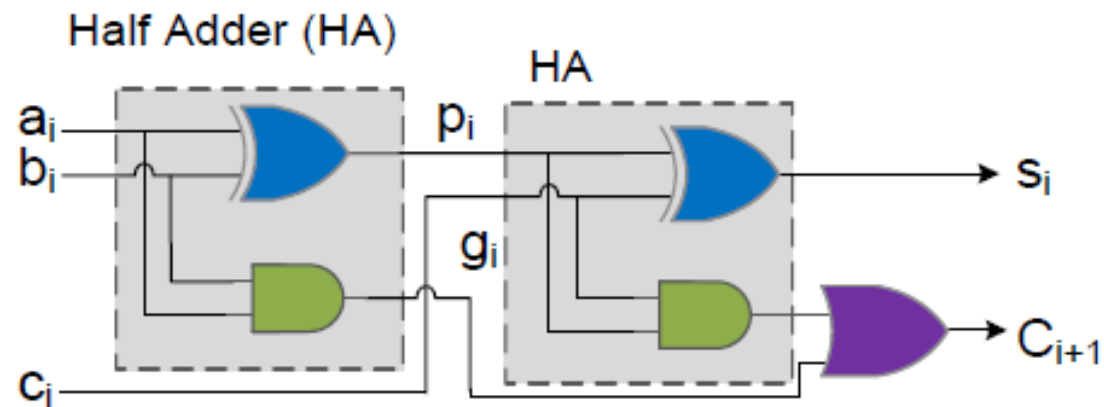
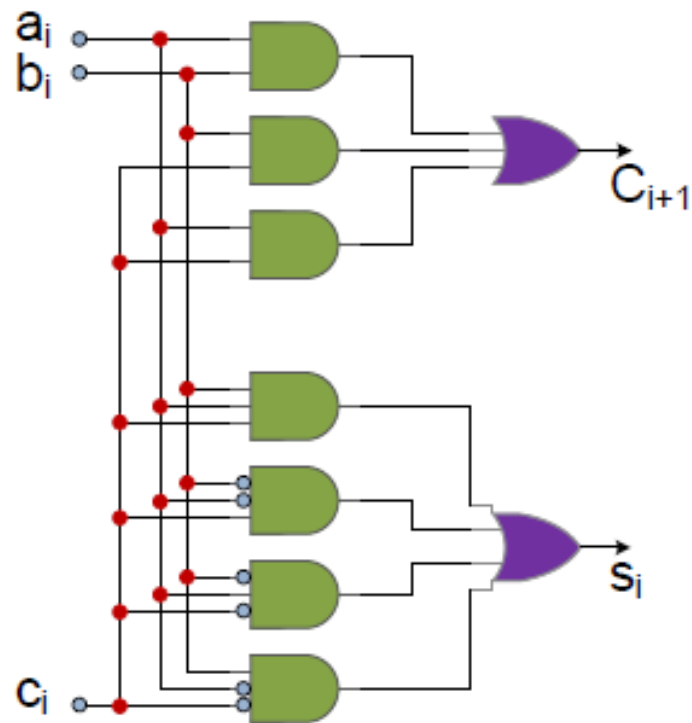
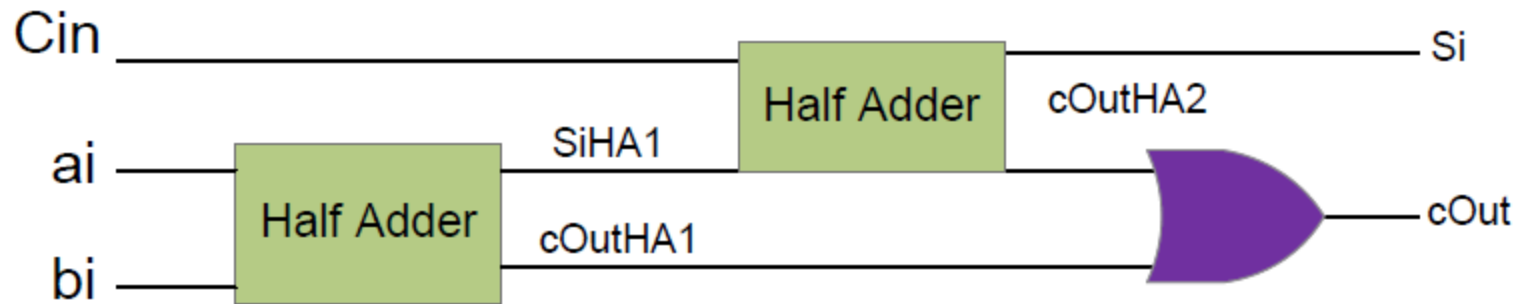
Adders

- Adders are used in addition, subtraction, multiplication and division.
- The speed of any digital design of a signal processing or communication system depends heavily on these functional units.
- The ripple carry adder (RCA) is the slowest in adder family.
- To cater for the slow carry propagation, fast adders are designed. These make the process of carry generation and its propagation faster.

Fast adders

- Carry look ahead adder
- Conditional sum adder
- Carry select adder
- Hierarchical Carry select adder

Single bit full adder/ Gate level options



Ripple carry adder

- Slowest adder due to carry propagation delay

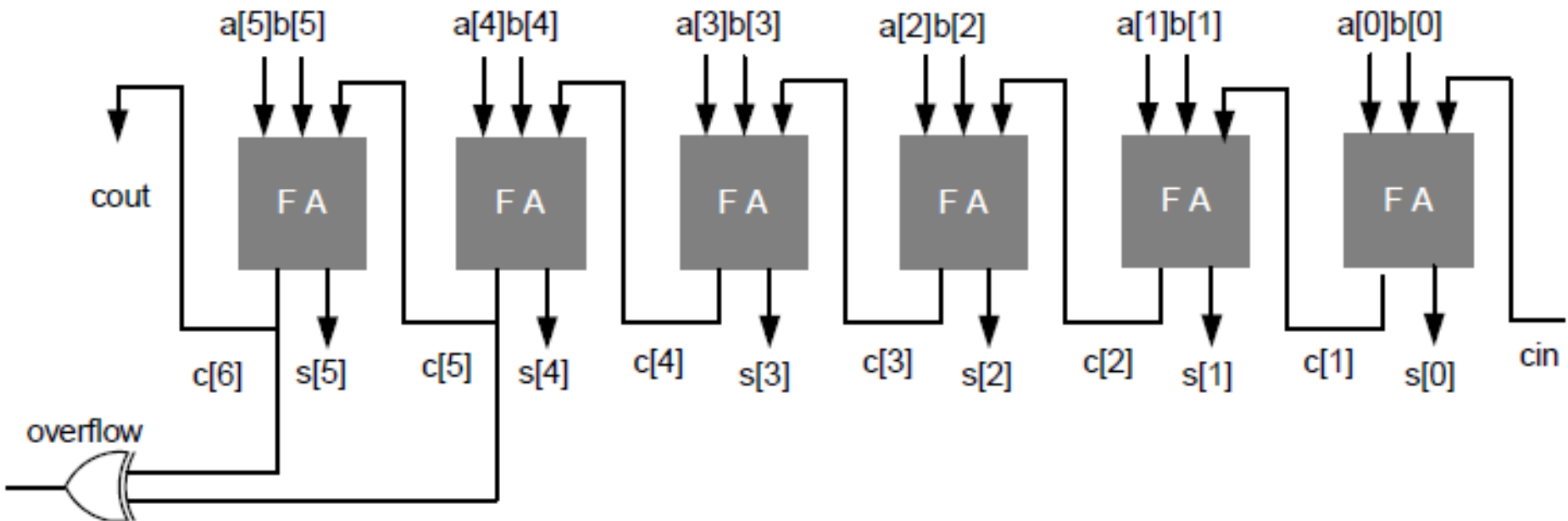


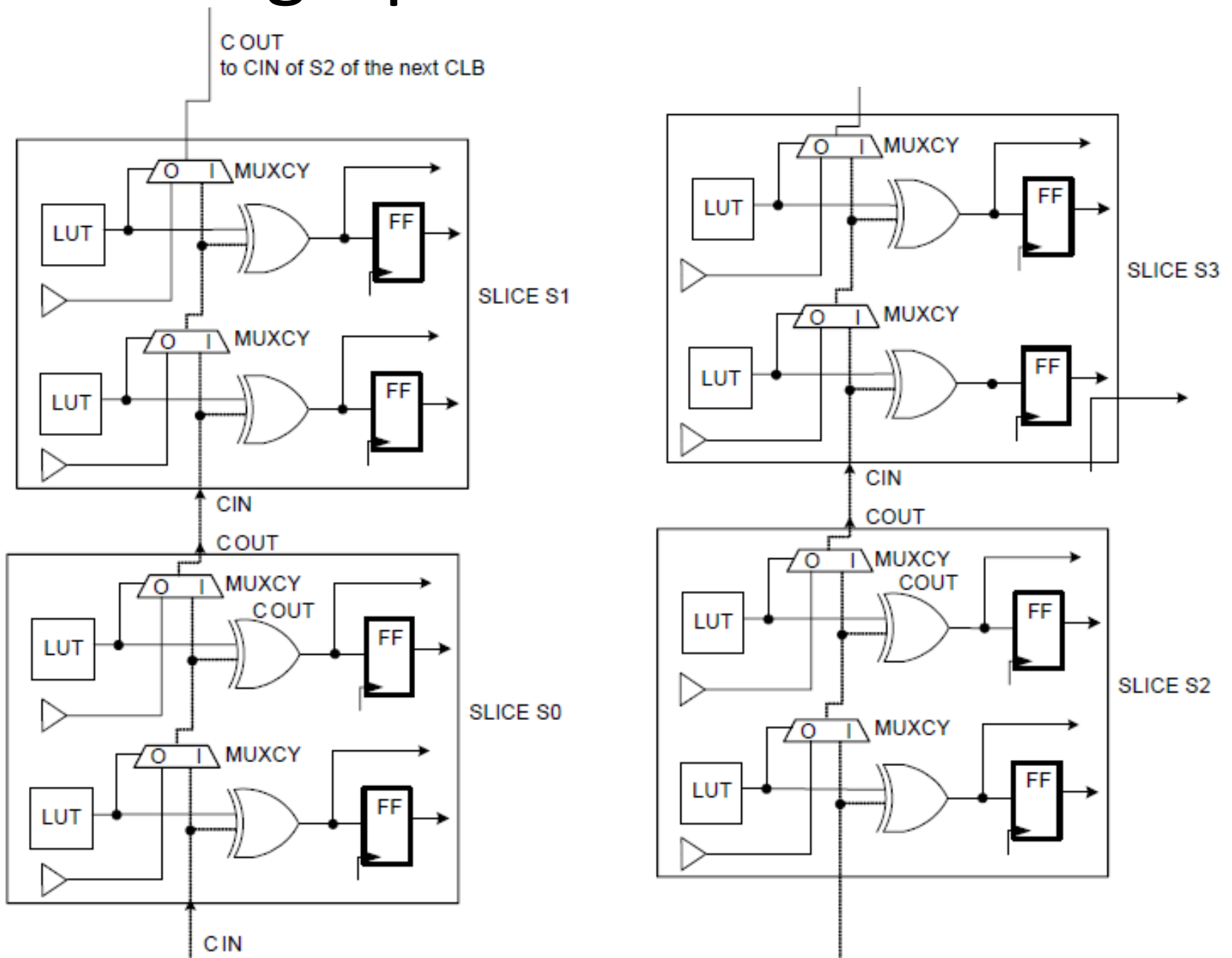
Figure 5.8 A 6 bit ripple carry adder

$$c_{i+1} = g_i + p_i c_i$$

$$p_i = a_i \oplus b_i$$

$$g_i = a_i b_i$$

Logic placement in CLBs



Carry look ahead adder

- A simple consideration of full adder logic identifies that a carry c_{i+1} is generated if $a_i = b_i = 1$, and a carry is propagated if either a_i or b_i is 1. This can be written as:

$$g_i = a_i b_i$$

$$p_i = a_i \oplus b_i$$

$$c_{i+1} = g_i + p_i c_i$$

$$s_i = c_i \oplus p_i$$

Carry look ahead adder

$$g_i = a_i b_i$$

$$p_i = a_i \oplus b_i$$

$$c_{i+1} = g_i + p_i c_i$$

$$s_i = c_i \oplus p_i$$

$$c_1 = g_0 + p_0 c_0$$

$$c_2 = g_1 + p_1 c_1$$

$$c_2 = g_1 + p_1(g_0 + p_0 c_0)$$

$$c_2 = g_1 + p_1 g_0 + p_0 p_1 c_0$$

$$c_i = g_{i-1} + \sum_{j=0}^{i-2} \left(\prod_{k=j+1}^{i-1} p_k \right) g_j + \prod_{k=0}^{i-1} p_k c_0$$

$$c_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0$$

$$c_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0$$

CLA logic

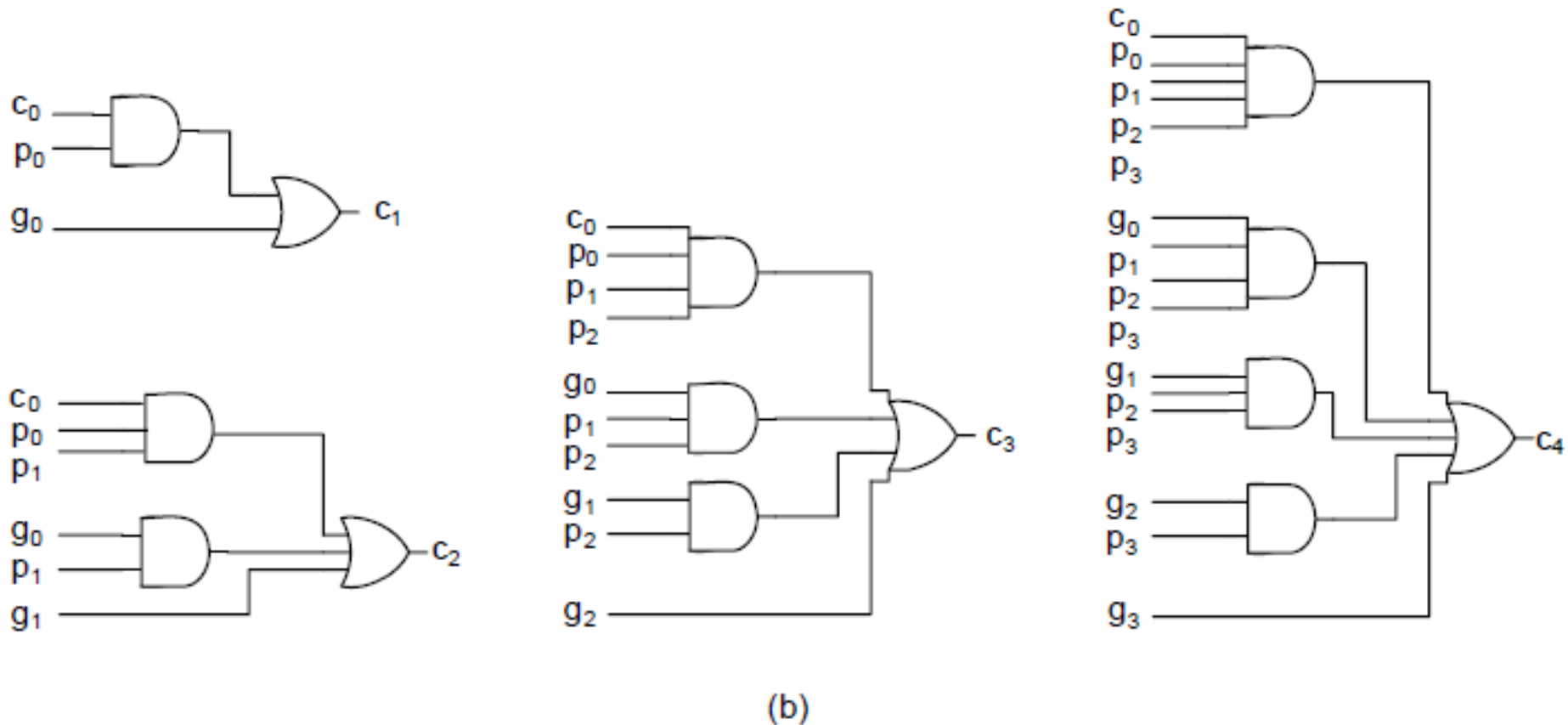


Figure 5.10 CLA logic for computing carries in two gate delay time

Grouping of CLAs

- Industrial practice is to use 4 bit wide blocks. This limits the computation of carries until c_3 , and c_4 is not computed. The first four terms in c_4 are grouped as G_0 and the product $p_3p_2p_1p_0$ in the last term is tagged as P_0 as given here

$$c_4 = g_3 + p_3g_2 + p_3p_2g_1 + p_3p_2p_1g_0 + p_3p_2p_1p_0c_0$$

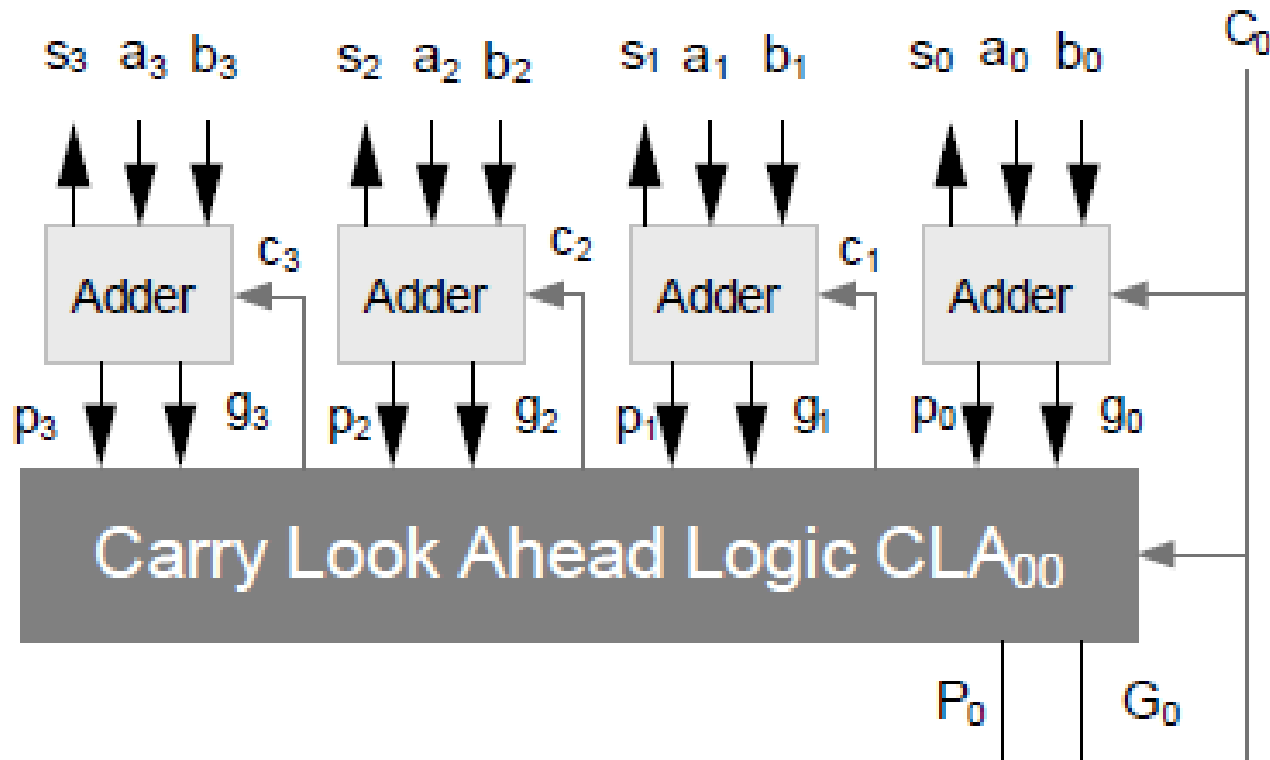
$$G_0 = g_3 + p_3g_2 + p_3p_2g_1 + p_3p_2p_1g_0$$

$$P_0 = p_3p_2p_1p_0$$

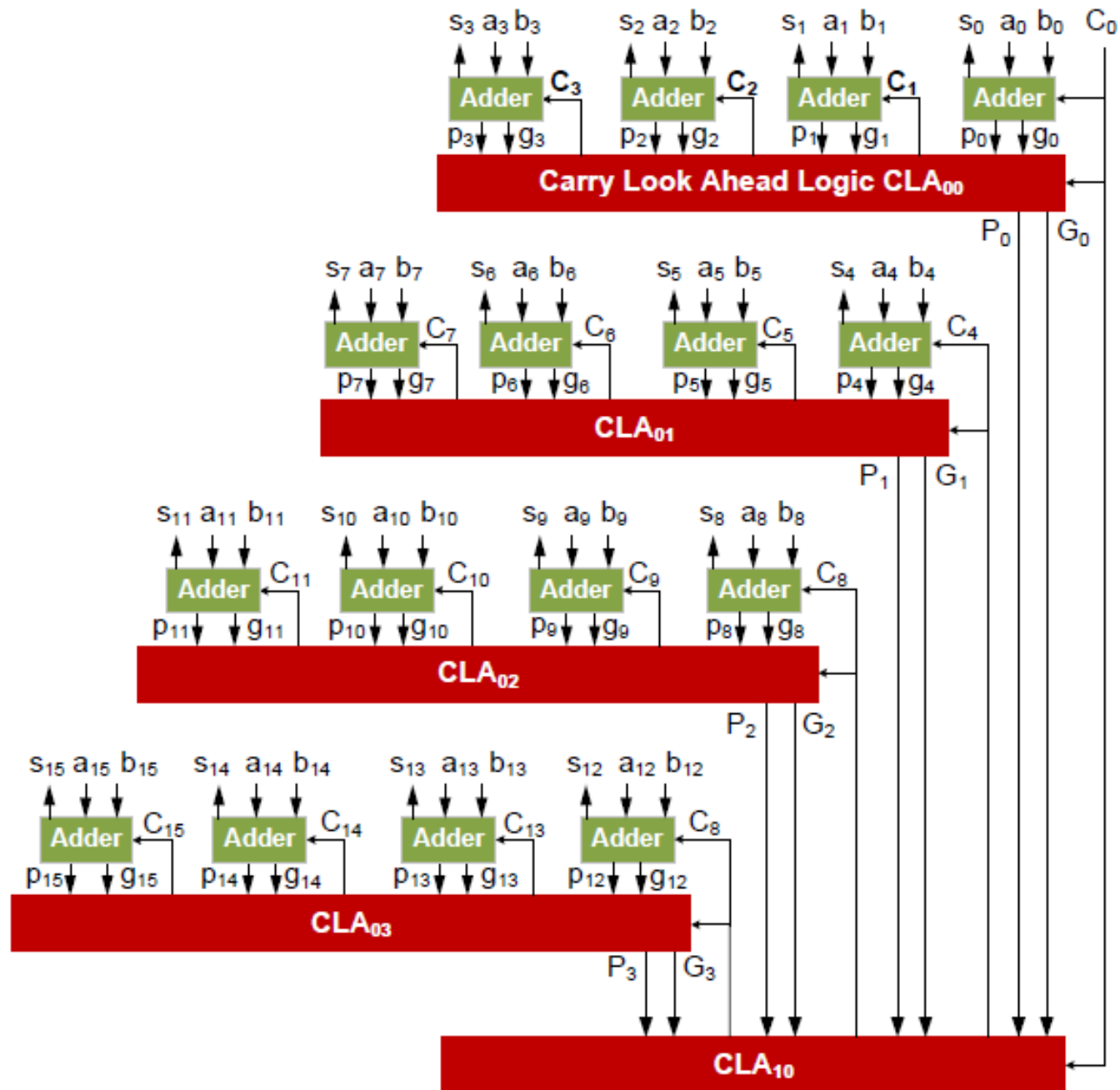
$$c_4 = G_0 + P_0c_0$$

Grouping of CLAs

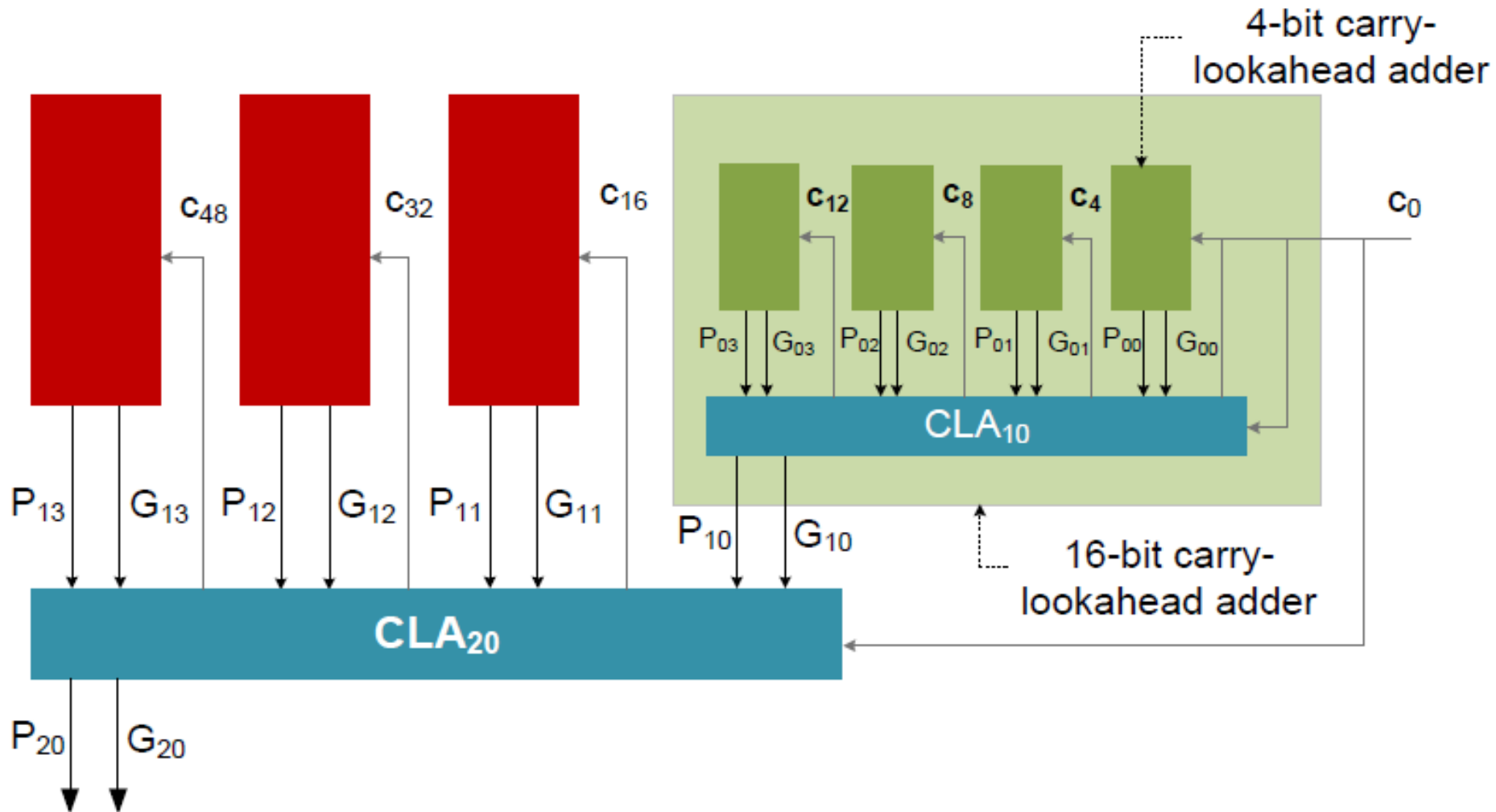
- Similarly, bits 4 to 7 are also grouped together and c_5 , c_6 and c_7 are computed in the first level of the CLA block using c_4 from the second level of CLA logic. The first level CLA block for these bits also generates G_1 and P_1 .



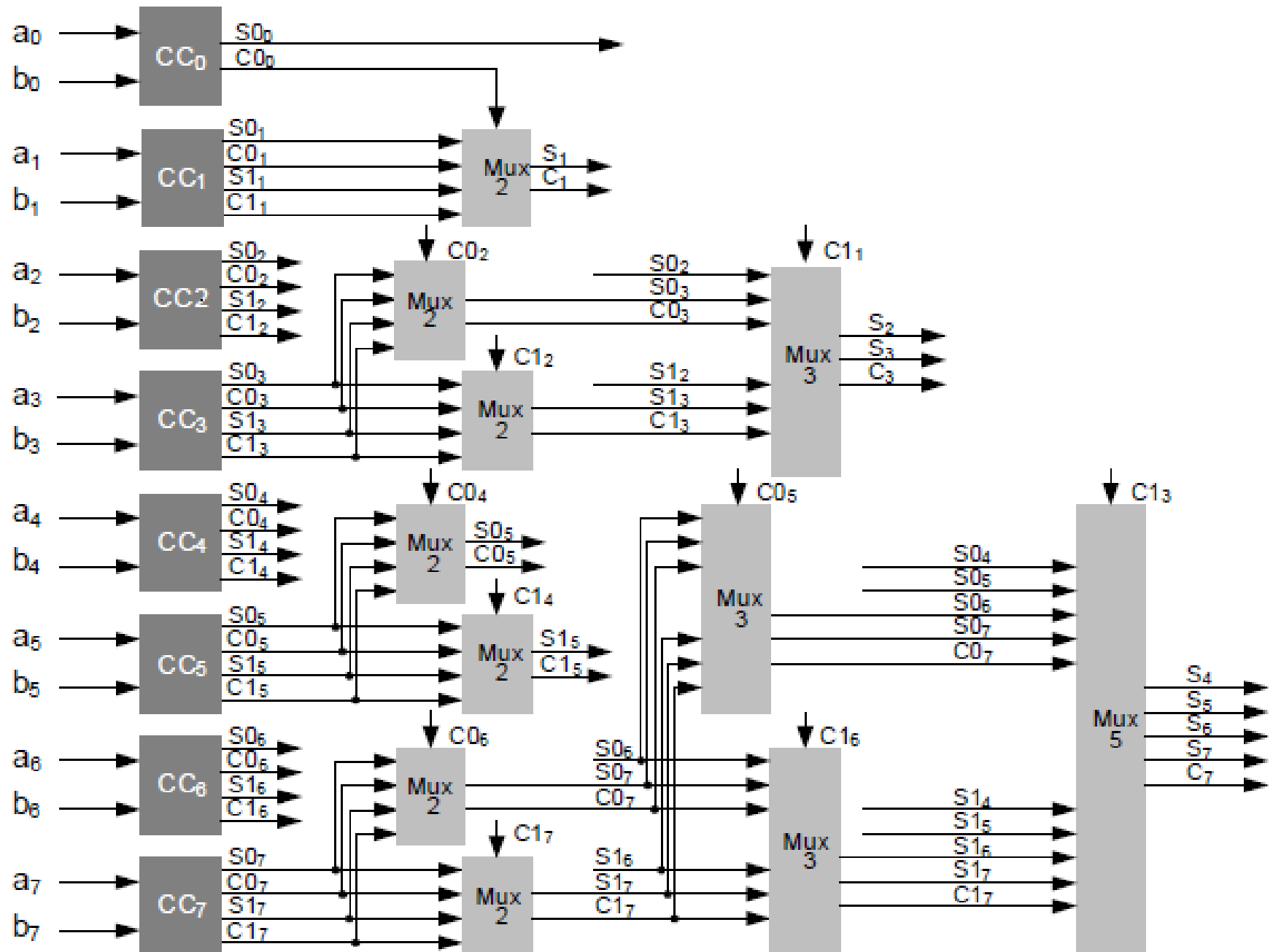
A 16-bit carry look-ahead adder using two levels of CLA logic



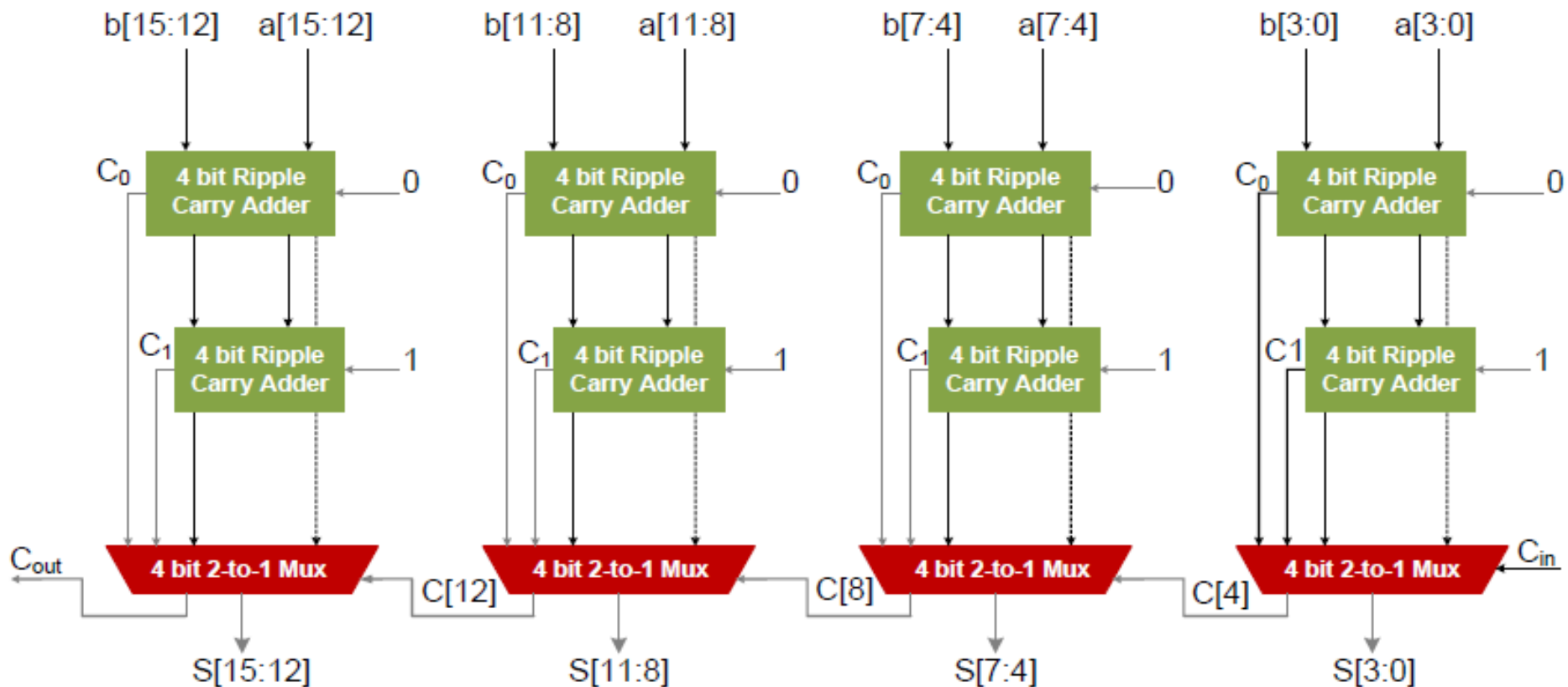
A 64-bit carry look-ahead adder using three levels of CLA logic



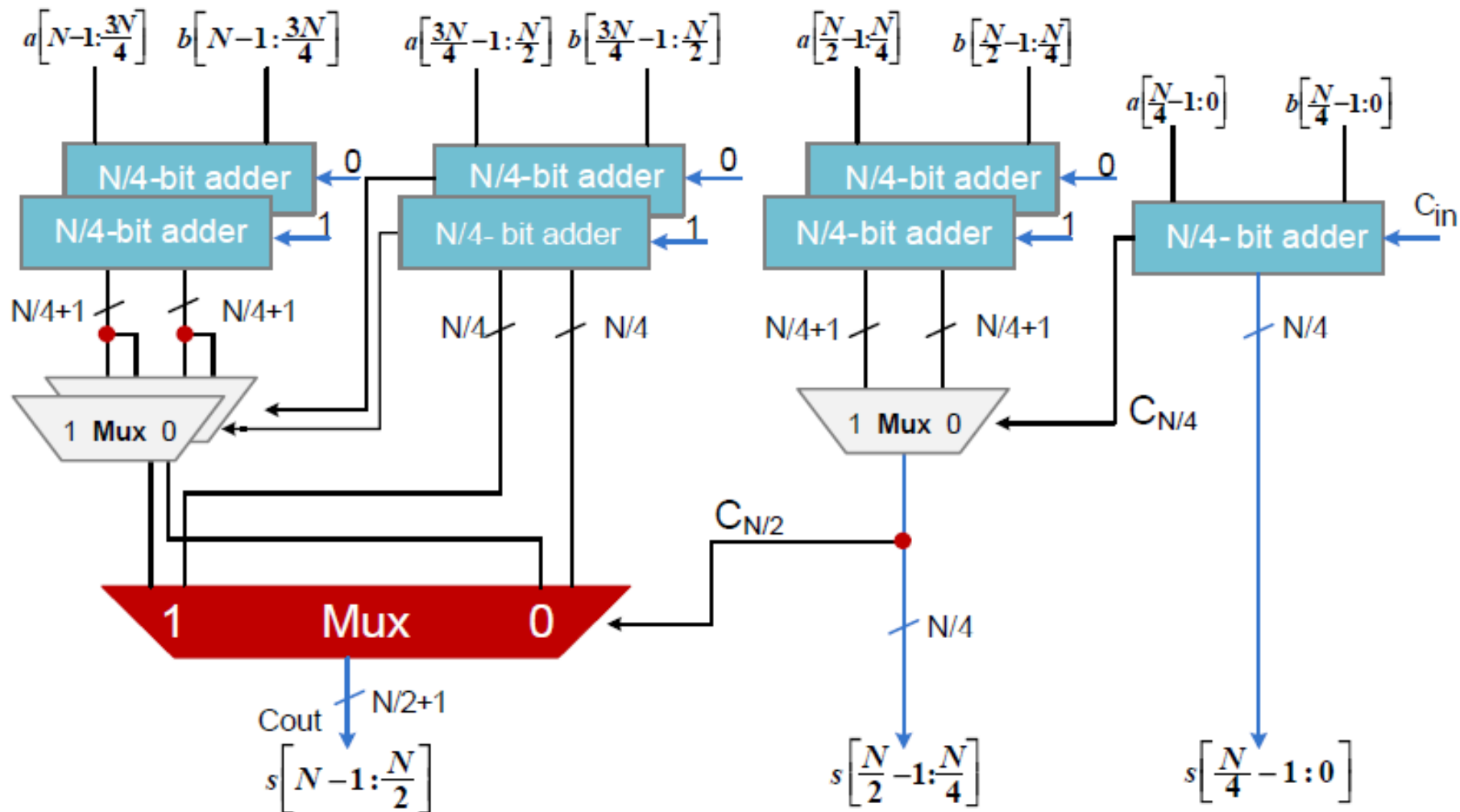
Conditional sum adder



16-bit uniform groups carry select adder



Hierarchical carry select adder

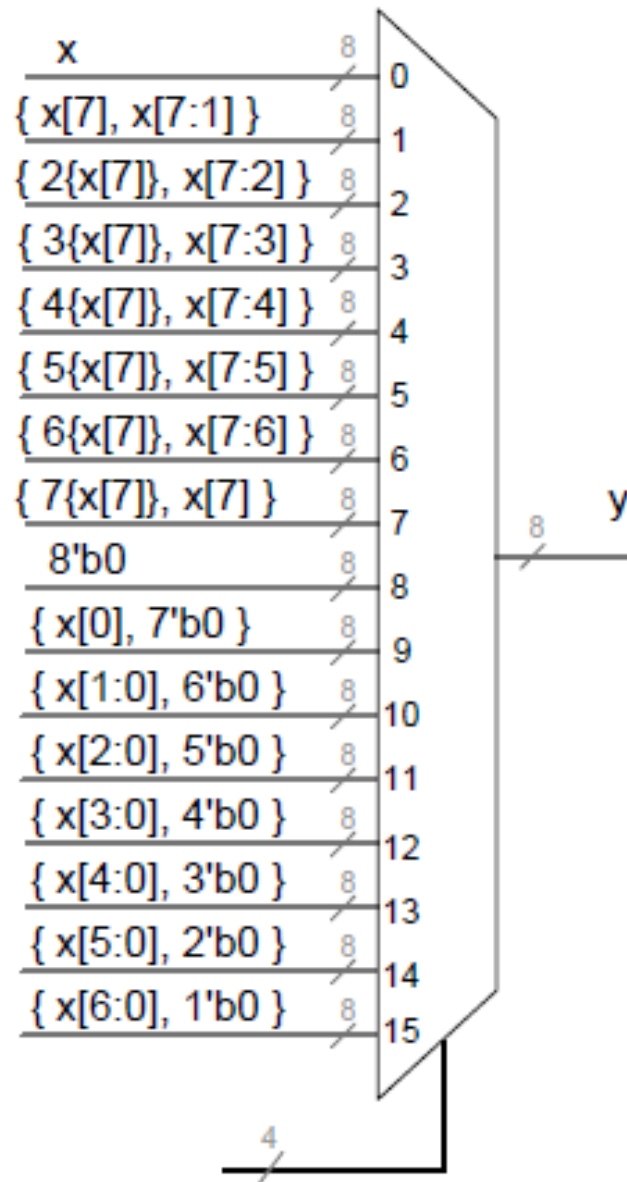


Barrel Shifter

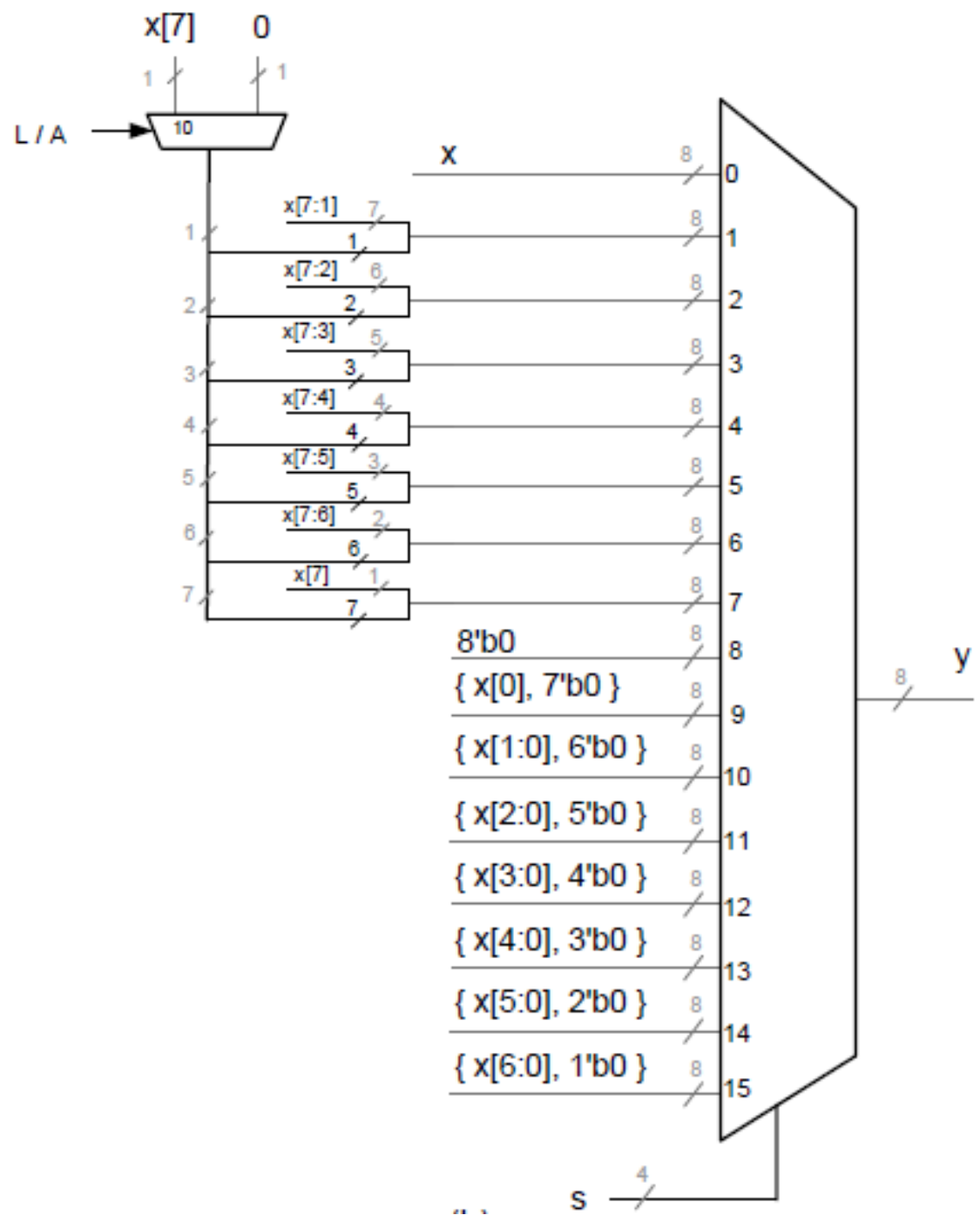
Barrel Shifter

- The circuit should support
 - Logical right shift: $x \gg S$
 - Logical left shift: $x \ll S$
 - Arithmetic right shift: $x \ggg S$
 - Arithmetic left shift: $x \lll S$

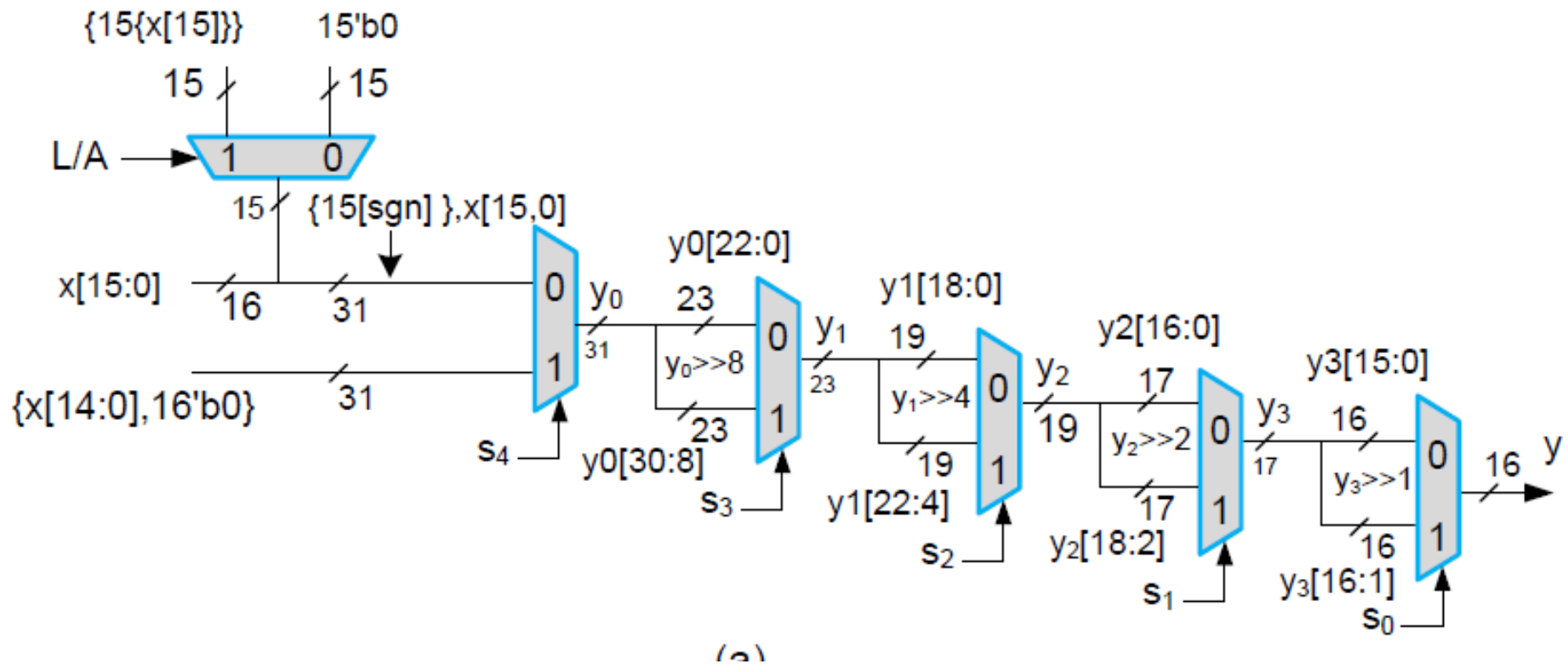
8-bit arithmetic shift



8-bit logical and arithmetic shift



Multi-stage barrel shifter

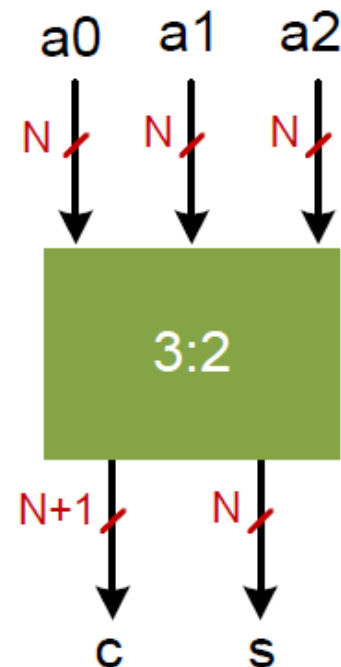


Parallel Multipliers

Carry Save Addition

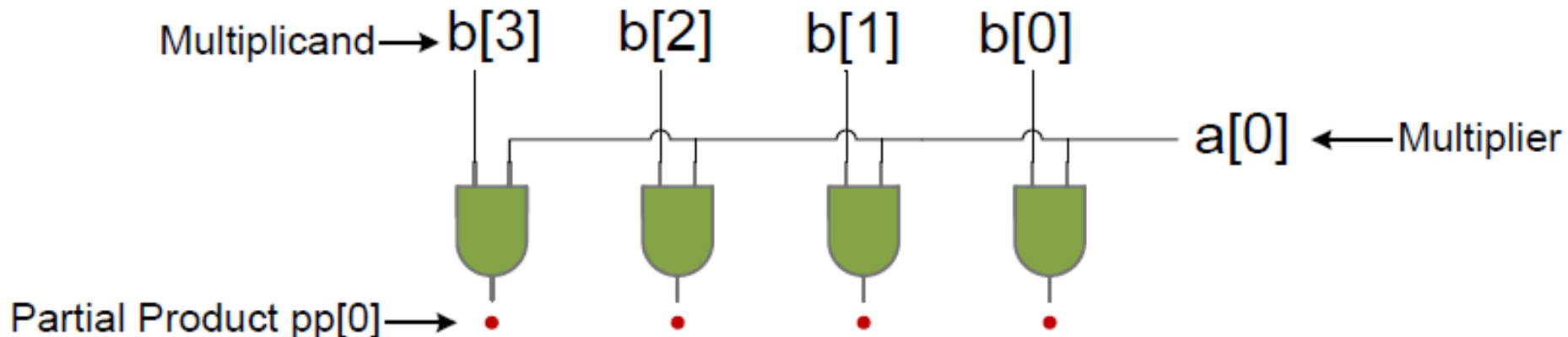
- while reducing three operands to two, does not propagate carries; rather, a carry is saved to the next significant bit position. Thus this addition reduces three operands to two without carry propagation delay

a0	=		0	0	1	0	1	1
a1	=		0	1	0	1	0	1
a2	=		1	1	1	1	0	1
s	=		1	0	0	0	1	1
c	=		0	1	1	1	0	1



Dot notation

- Dot notation facilitates description of different reduction schemes
- Dots are used to represent each bit of the partial product



Parallel multiplier circuits

- A CSA is one of the fundamental building blocks of most parallel multiplier architectures. The partial products are first reduced to two numbers using a CSA tree. These two numbers are then added to get the final product.

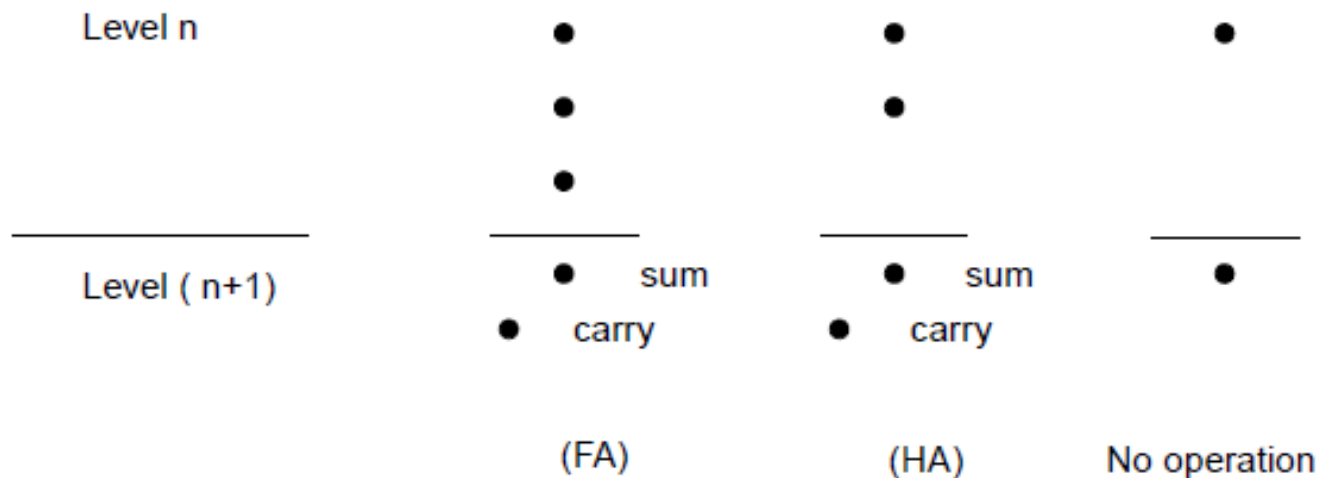
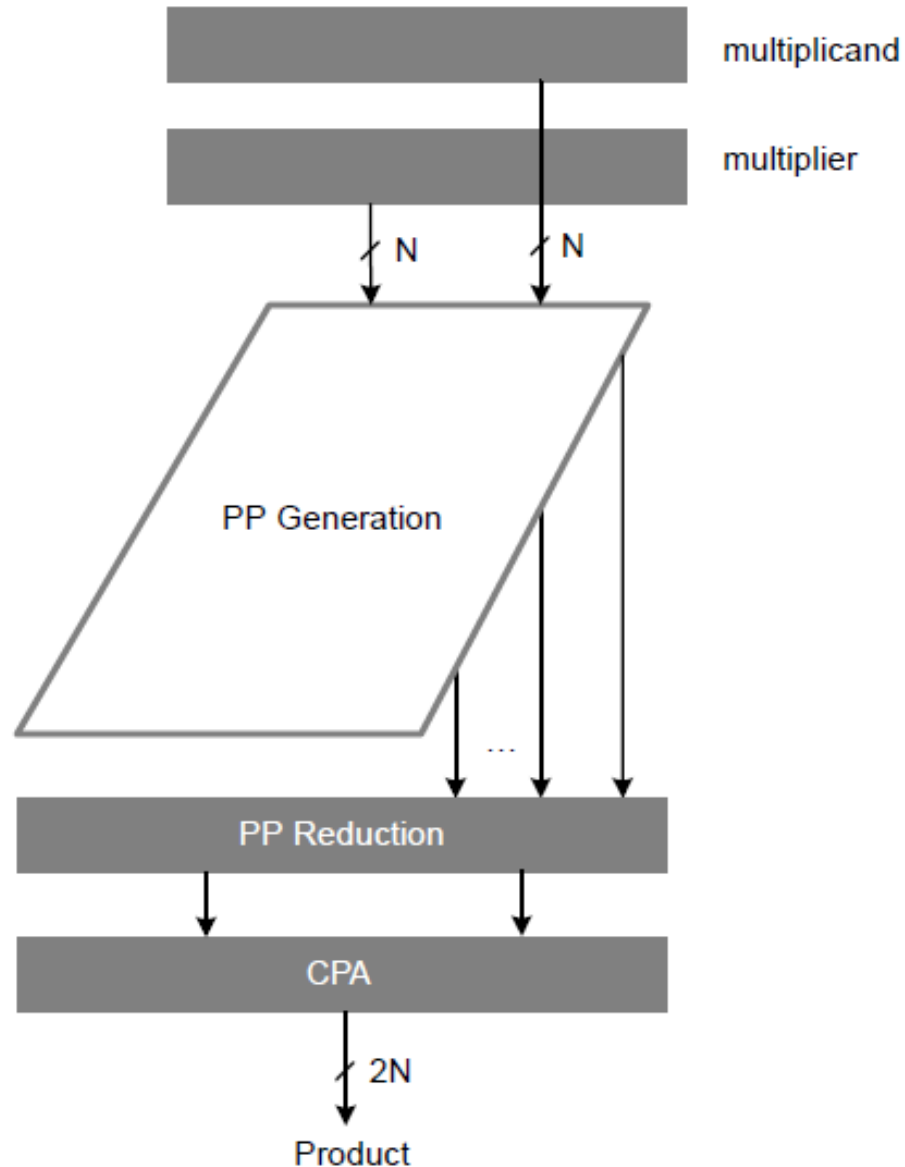


Figure 5.26 Reducing the number of dots in a column

Three components of a multiplier



Partial Product Generation

- Partial products $PP[i]$ are generated by ANDing each bit $a(i)$ of the multiplier with all the bits of the multiplicand b
- Each $PP[i]$ is shifted to the left by i bit positions

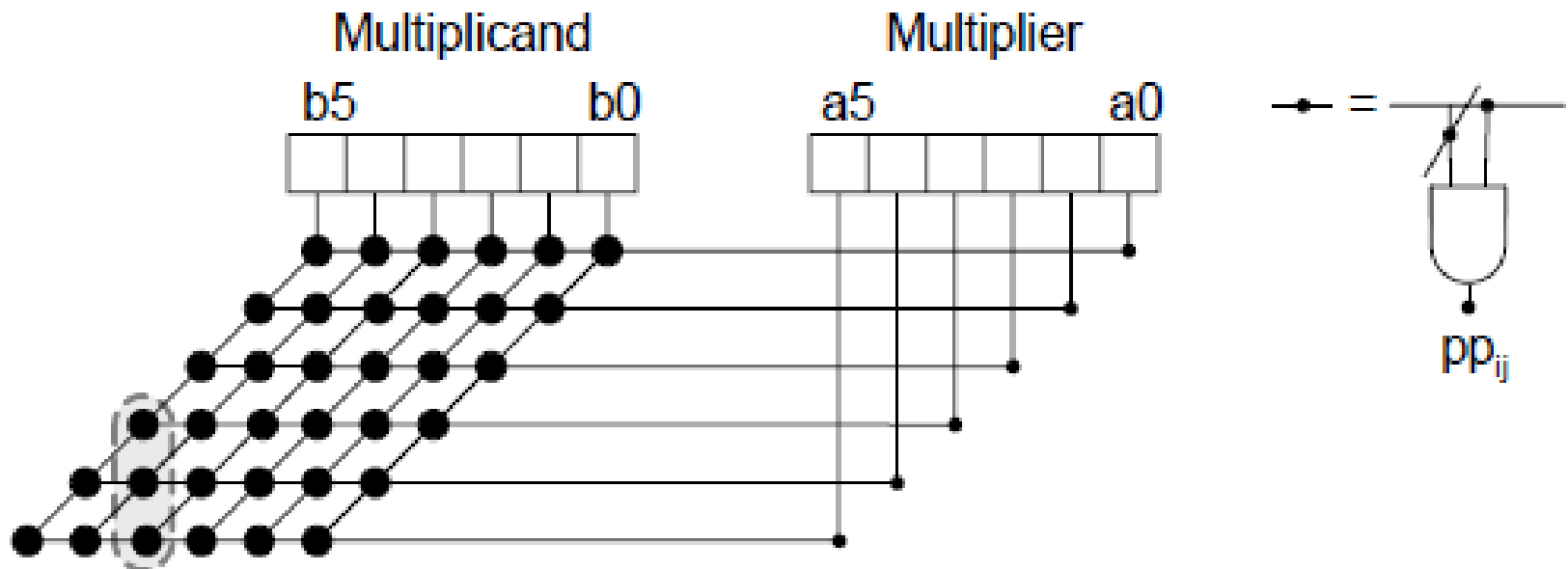


Figure 5.28 Partial product generation for a 6×6 multiplier

PPs generation code

```
module multiplier
(
input [5:0] a,b,
output [11:0] prod);
integer i;
reg [5:0] pp [0:5]; //6 partial products
always@*
begin
    for(i 0; i<6; i i+1)
        begin
            pp[i]  b & {6{a[i]}};
        end
end
    assign prod  pp[0]+{pp[1],1'b0}+{pp[2],2'b0} +
                {pp[3],3'b0}+{pp[4],4'b0}+{pp[5],5'b0};
endmodule
```

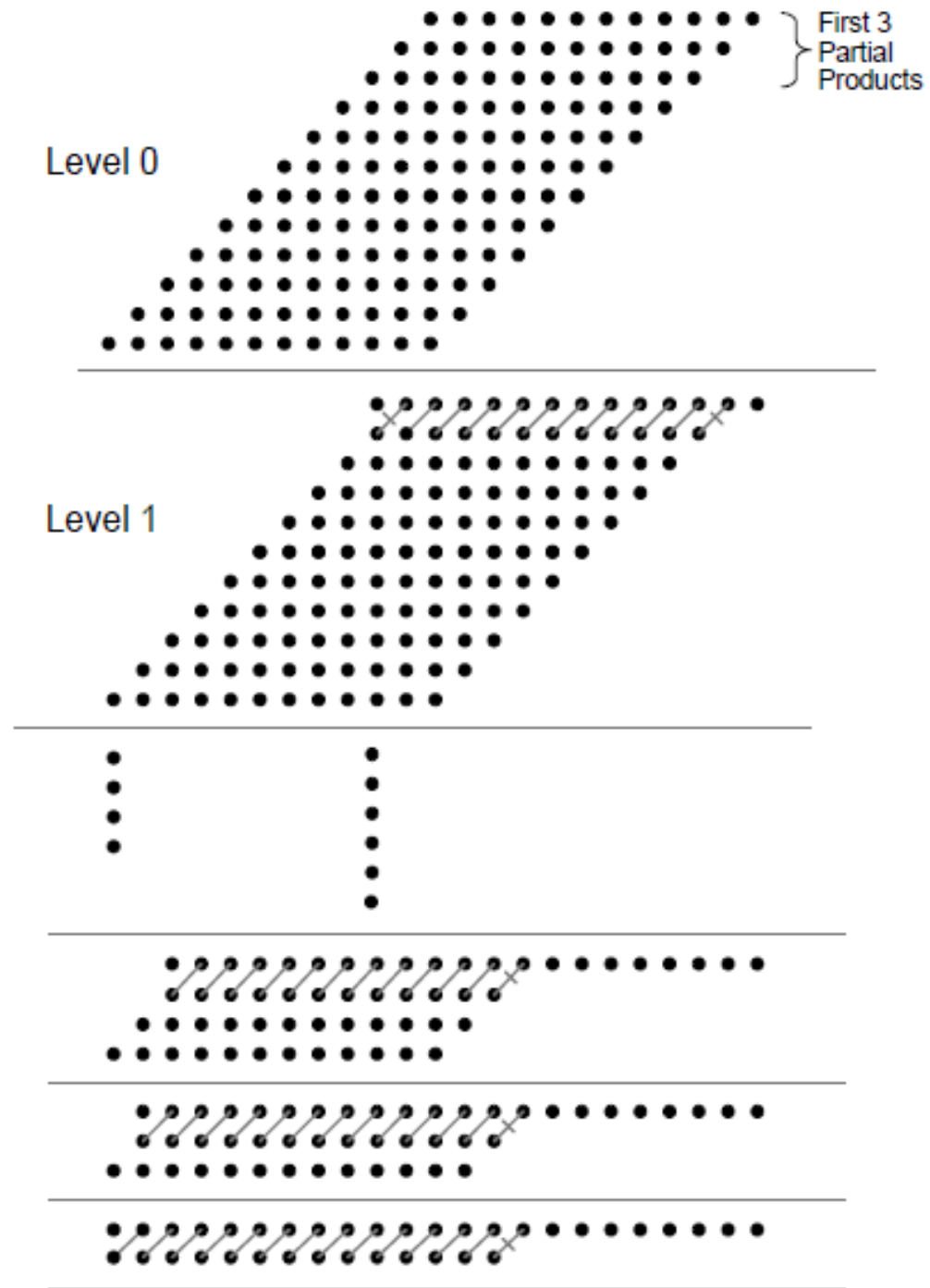
Partial Product Reduction

- For a general $N_1 \times N_2$ multiplier, the following four techniques are generally used to reduce N_1 layers of the partial products to two layers for their final addition using any CPA:
 - carry save reduction
 - dual carry save reduction
 - Wallace tree reduction
 - Dadda tree reduction.

Carry Save Reduction

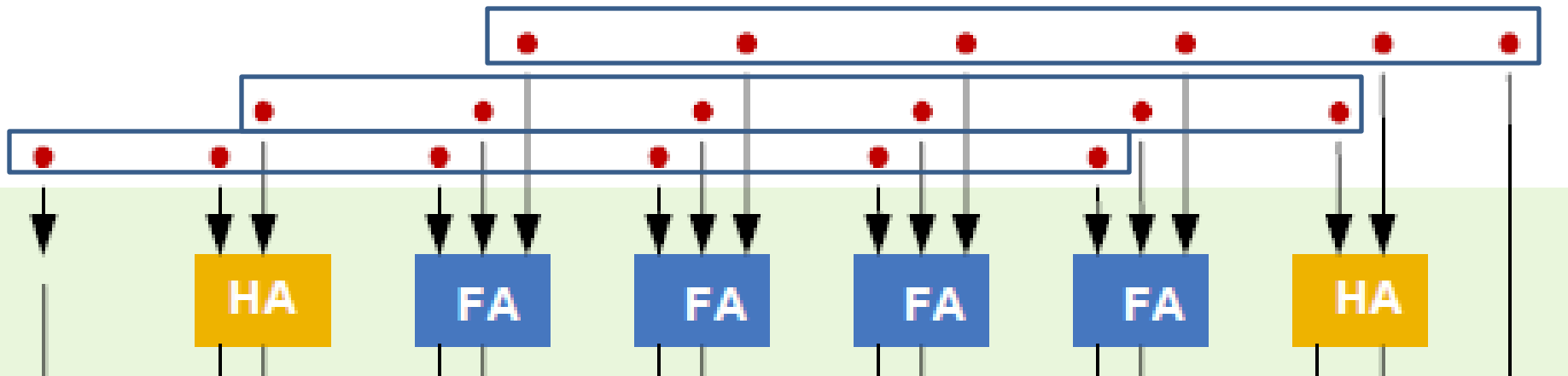
- The first three layers of the PPs are reduced to two layers using carry save addition (CSA).
- Isolated bits in a column three layers, are simply dropped down to the same column
- Columns with two bits are reduced to two bits using half adders and the columns with three bits are reduced to two bits using full adders
- Once the first three PPs are reduced to two layers, the fourth partial product is grouped with them to make a new group of three layers.
- The process is repeated until two layers are left which are added using CPA

12x12 multiplier PPs reduction



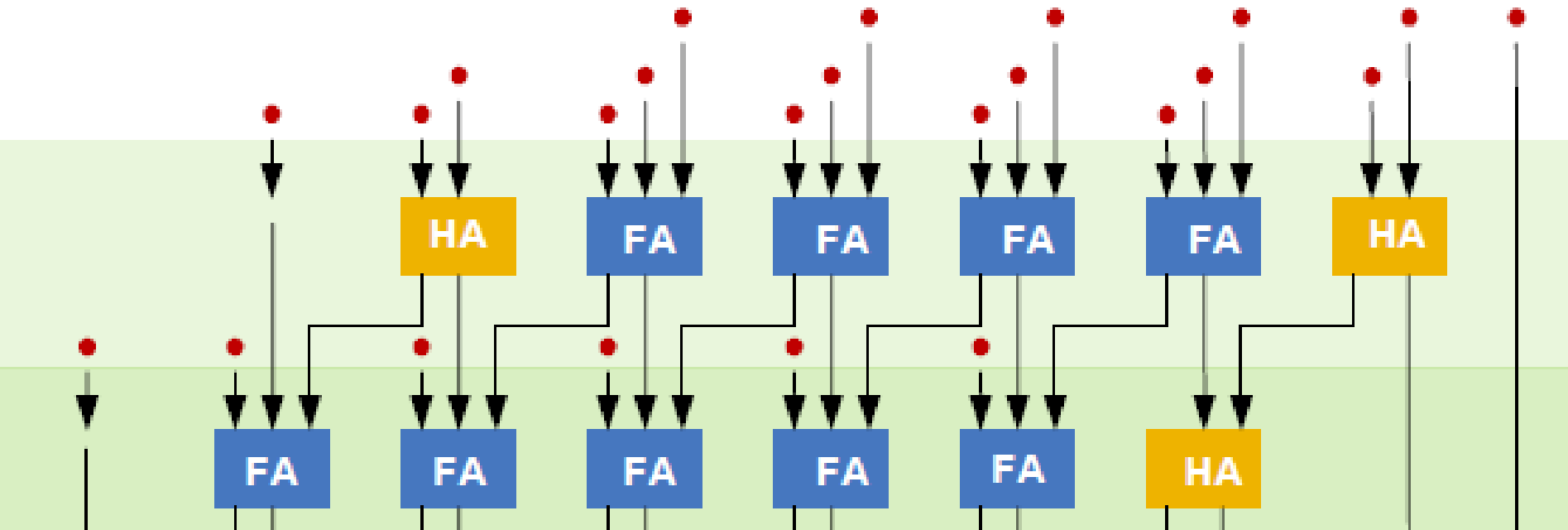
Carry save reduction scheme layout for a 6x6 multiplier

- Level 0

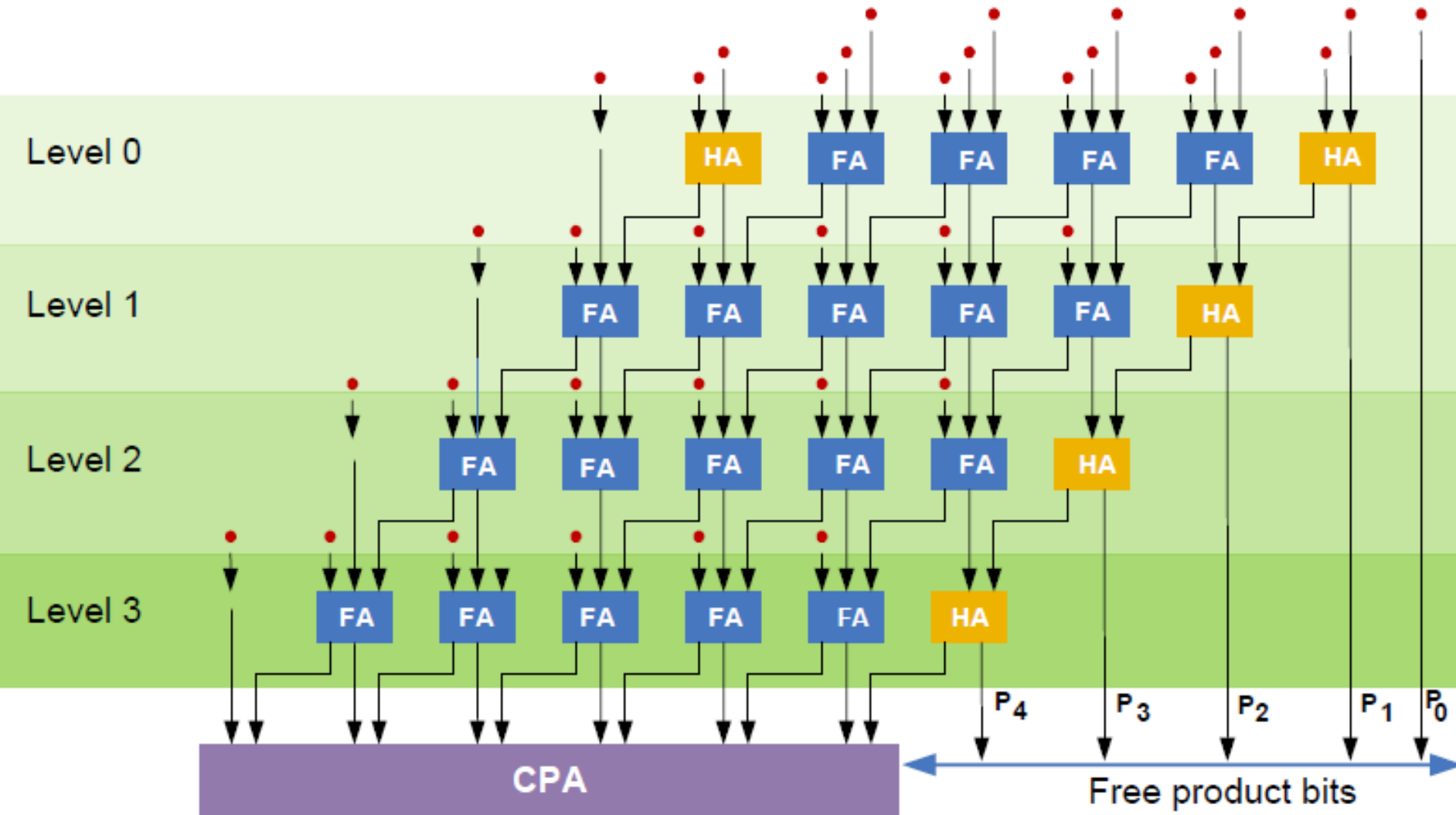


Carry save reduction scheme layout for a 6x6 multiplier

- Level 1



Carry save reduction scheme layout for a 6x6 multiplier



Tree diagram 6x6

Tree diagram

Dual Carry Save Reduction

- The partial products are divided into 2 equal size groups
- The carry save reduction scheme is applied on both the groups simultaneously
- This results into two partial product layers in each group
- The four layers are then reduced using Carry Save Reduction
- The last two layers are added using any CPA

Tree diagram 8x8

Tree diagram

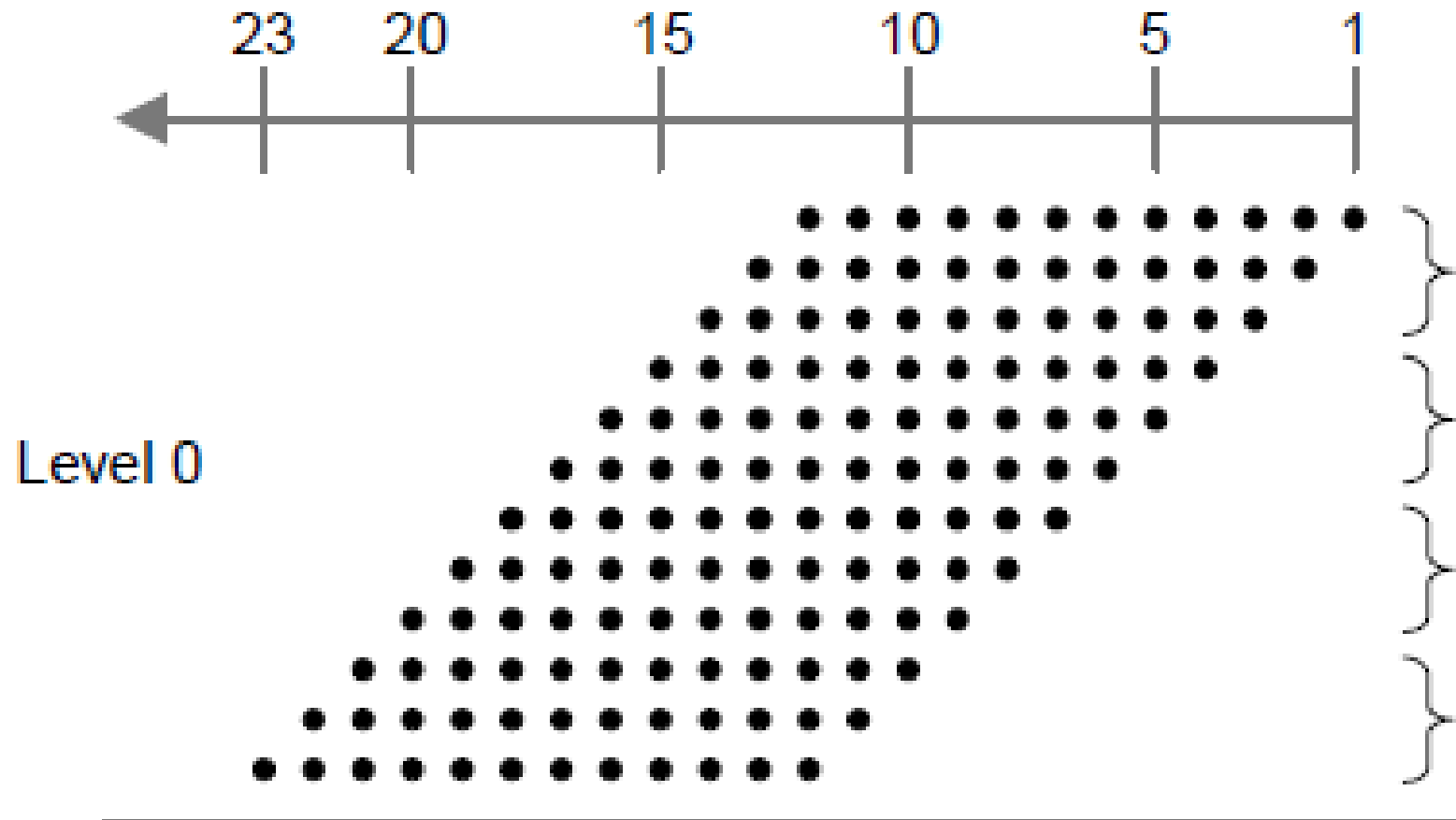
Wallace Tree Multipliers

- One of the most commonly used multiplier architecture
- The number of adder levels increase logarithmically as the partial products increase

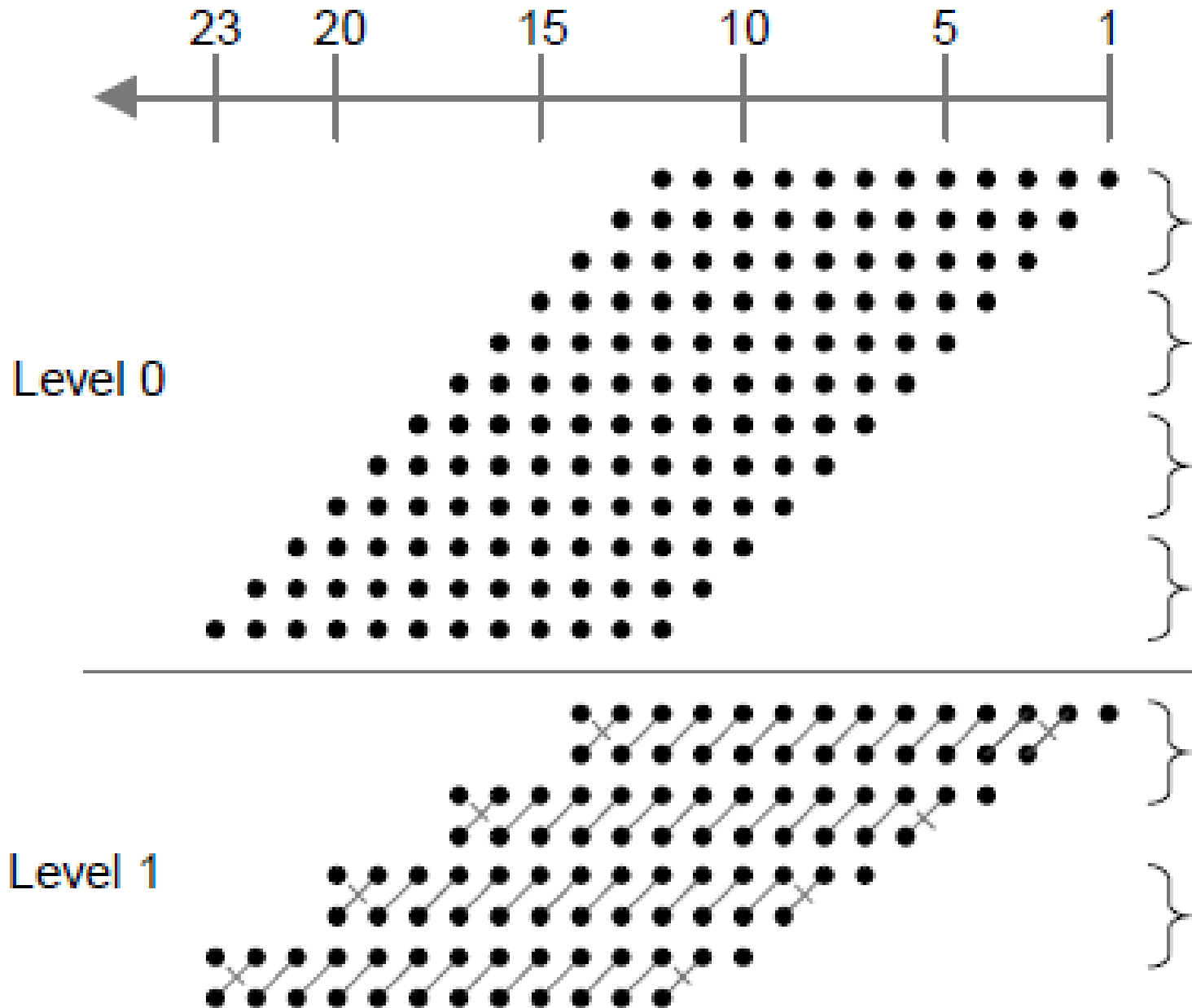
Wallace Tree Multipliers

- Make group of threes and apply CSA reduction in parallel
- Each CSA layer produces two rows
- Repeat the above two steps until two rows are left
- The final rows are added together using CPA for the final product

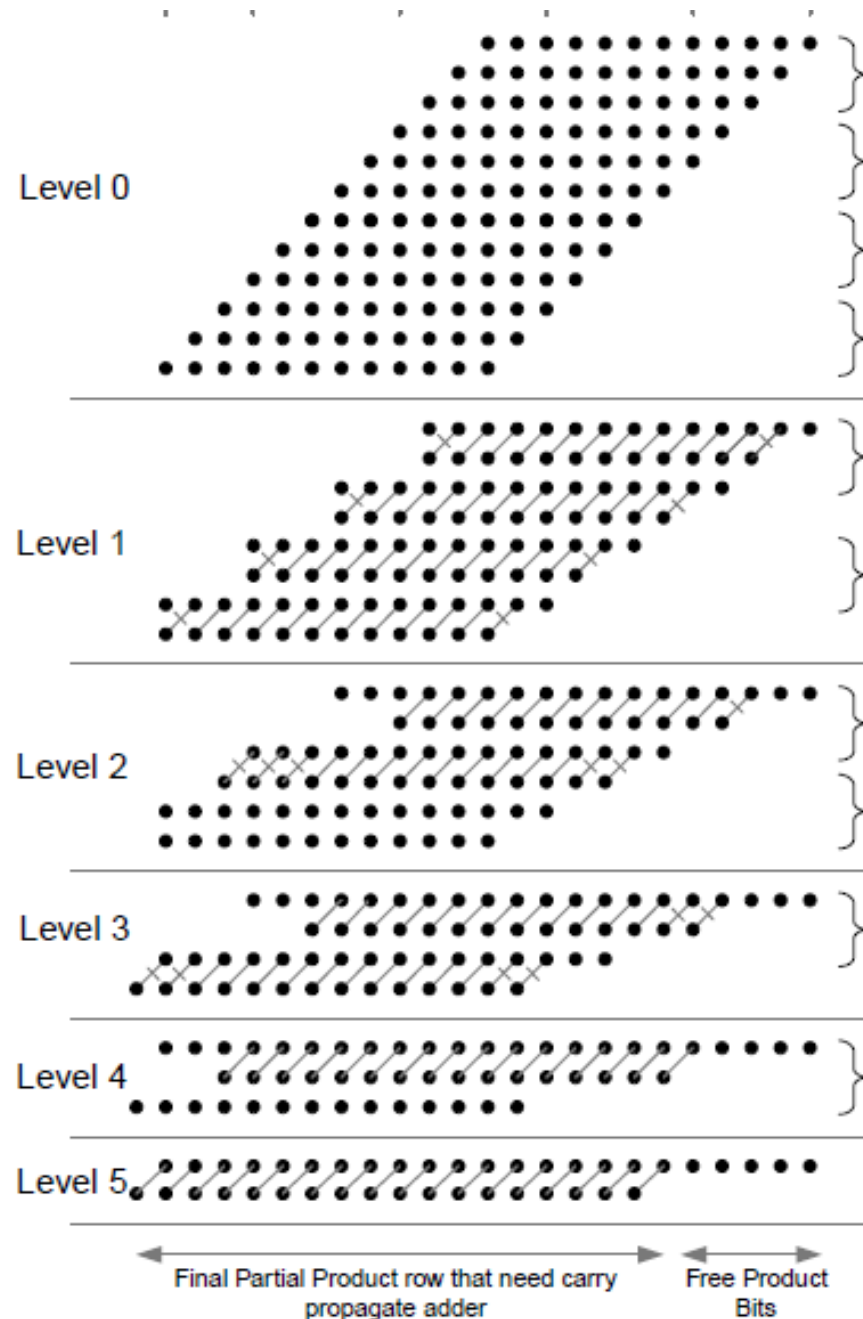
12x12 multiplication using Wallace Reduction Tree



12x12 multiplication using Wallace Reduction Tree



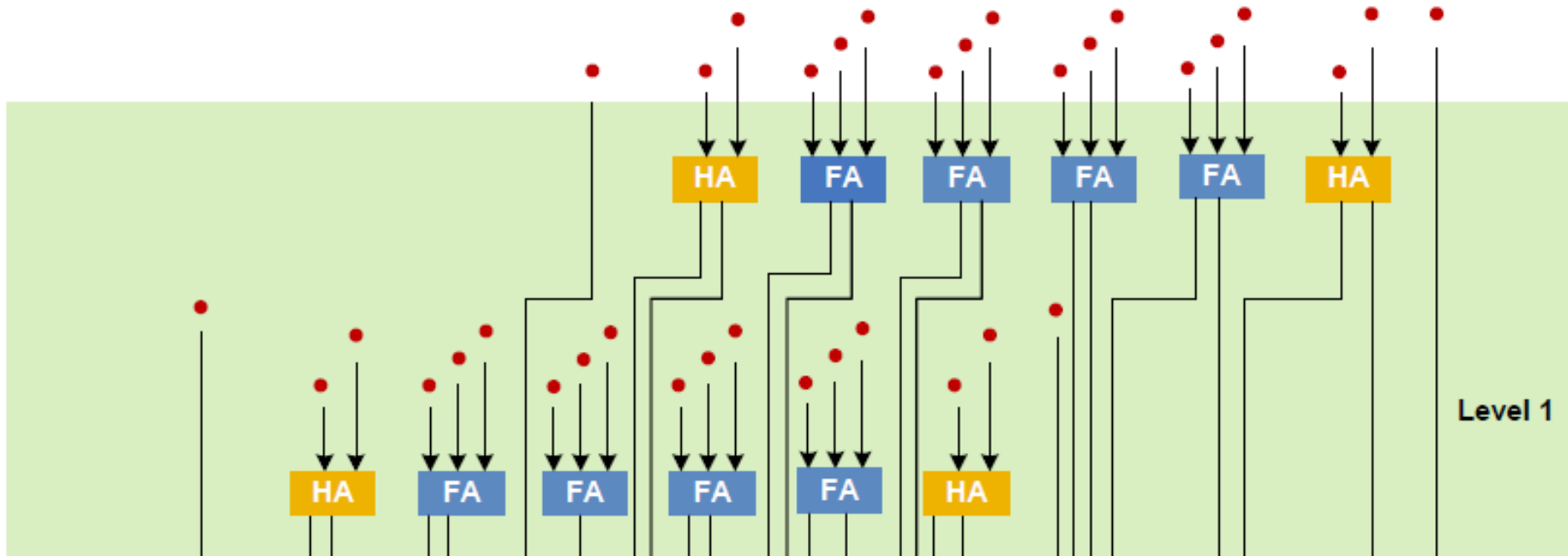
12x12 multiplication using Wallace Reduction Tree



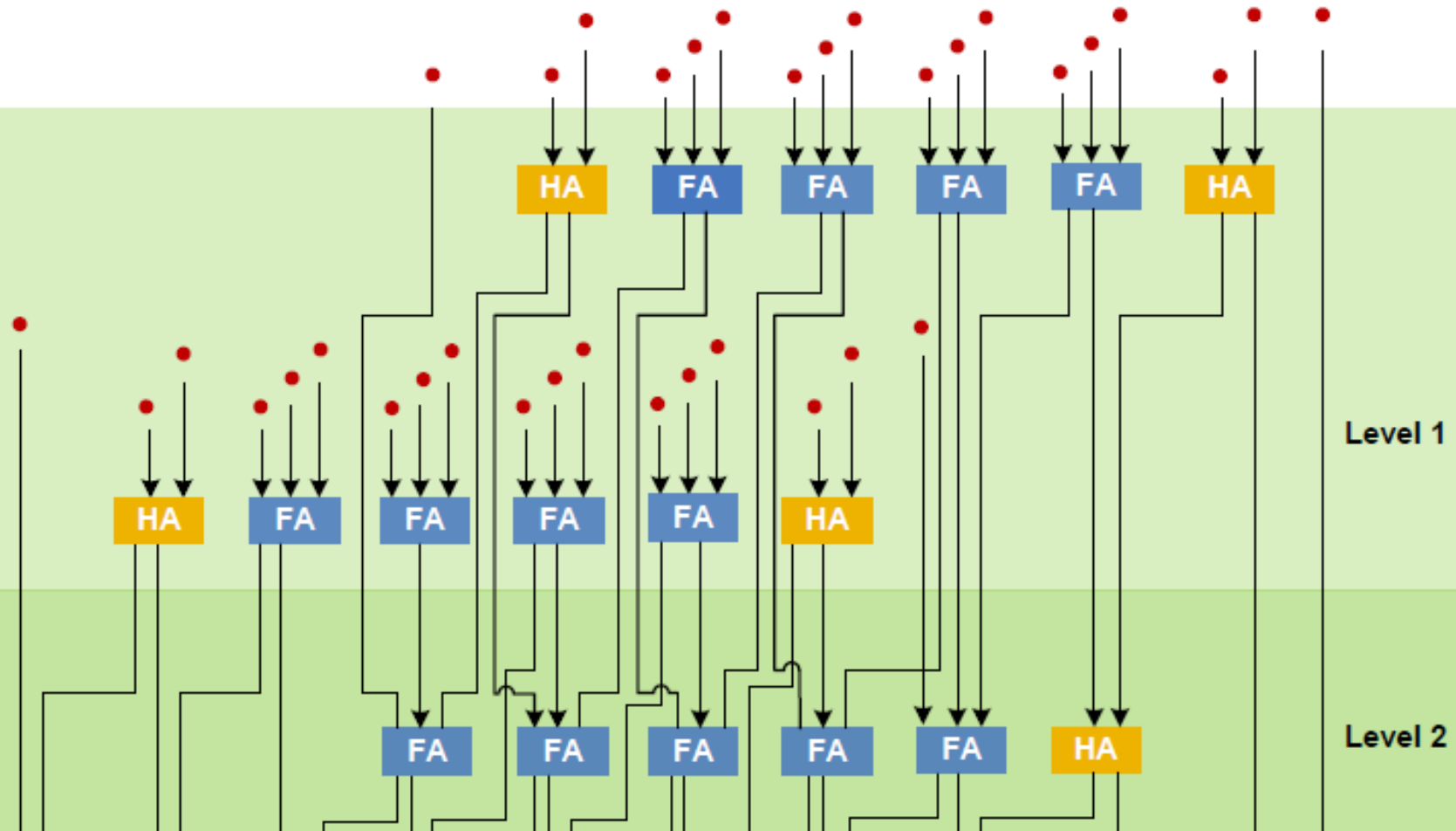
Wallace tree diagram 12x12

Wallace tree

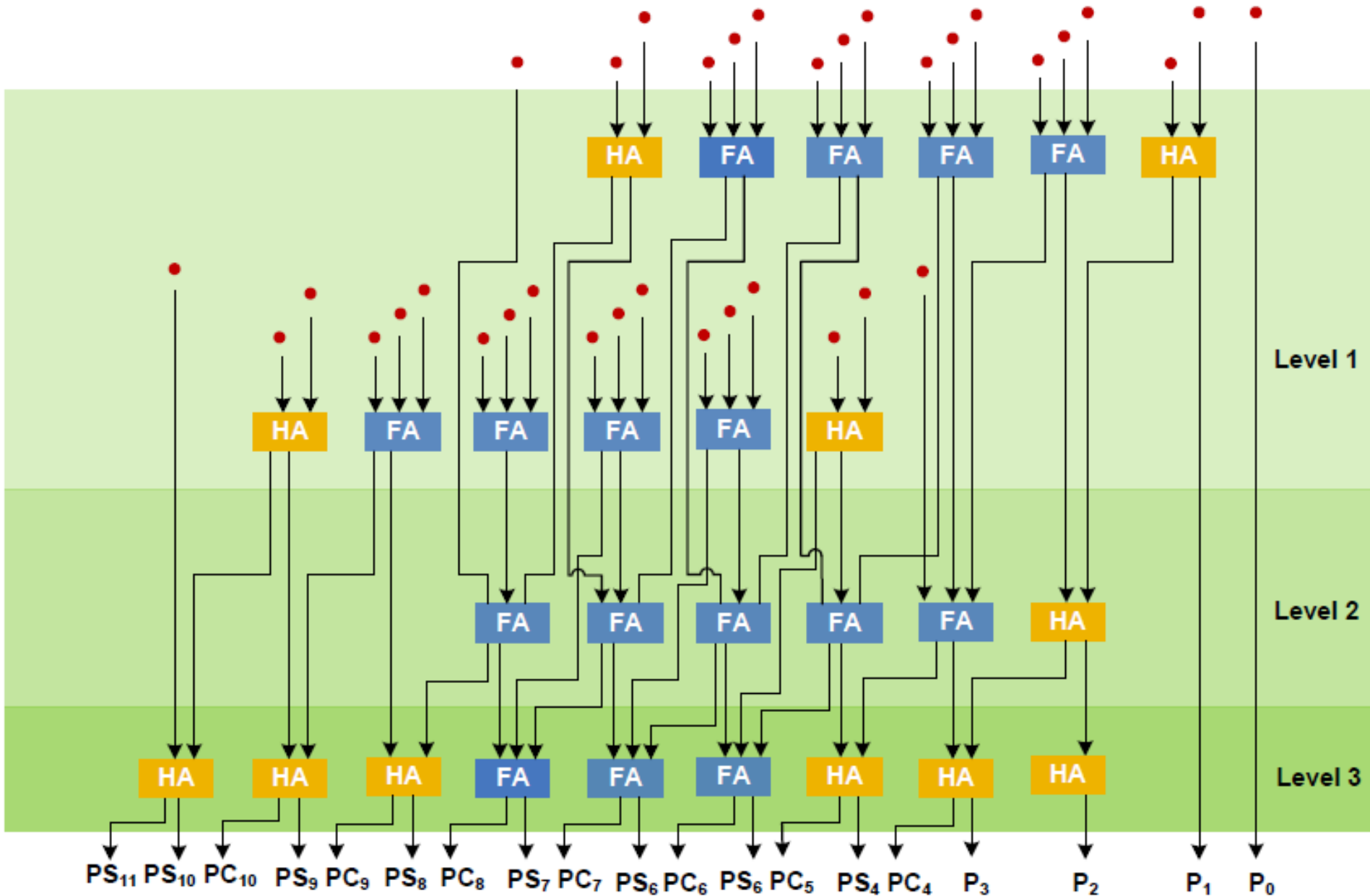
Wallace Reduction layout for a 6x6 array of PPs



Wallace Reduction layout for a 6x6 array of PPs



Wallace Reduction layout for a 6x6 array of PPs



Adder delays in Wallace tree

Table 5.3 Logarithmic increase in adder levels with increasing number of partial products

Number of PPs	Number of full adder delays
3	1
4	2
$5 \leq n \leq 6$	3
$7 \leq n \leq 9$	4
$10 \leq n \leq 13$	5
$14 \leq n \leq 19$	6
$20 \leq n \leq 28$	7
$29 \leq n \leq 42$	8
$43 \leq n \leq 63$	9

A Decomposed Multiplier

- Four Multipliers of size NxN can be combined to make a 2N x 2N multiplier

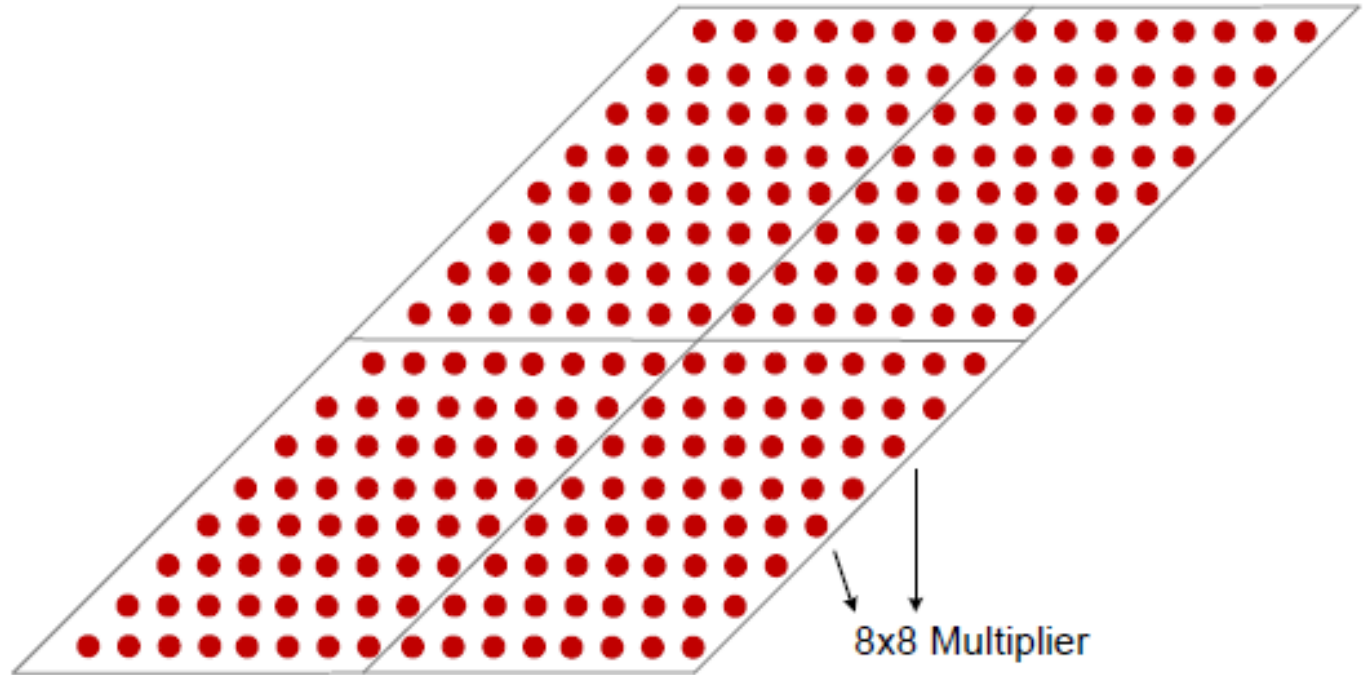
$$(a_L + 2^8 a_H) \times (b_L + 2^8 b_H)$$

$$= (a_L \times b_L + a_L b_H 2^8 + a_H b_L 2^8 + a_H b_H 2^{16})$$

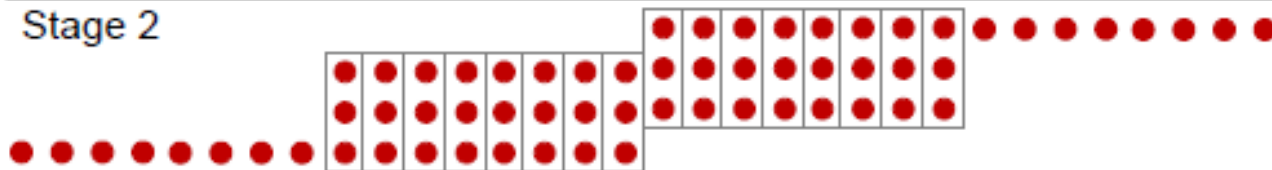
		$a_L \times b_L$ 16-Bits	
	$a_L \times b_H$ 16-Bits		
	$a_H \times b_L$ 16-Bits		
$a_H \times b_H$ 16-Bits			
32-Bits			

A 16x16 bit Multiplier decomposed into four 8x8 multipliers

Stage 1



Stage 2

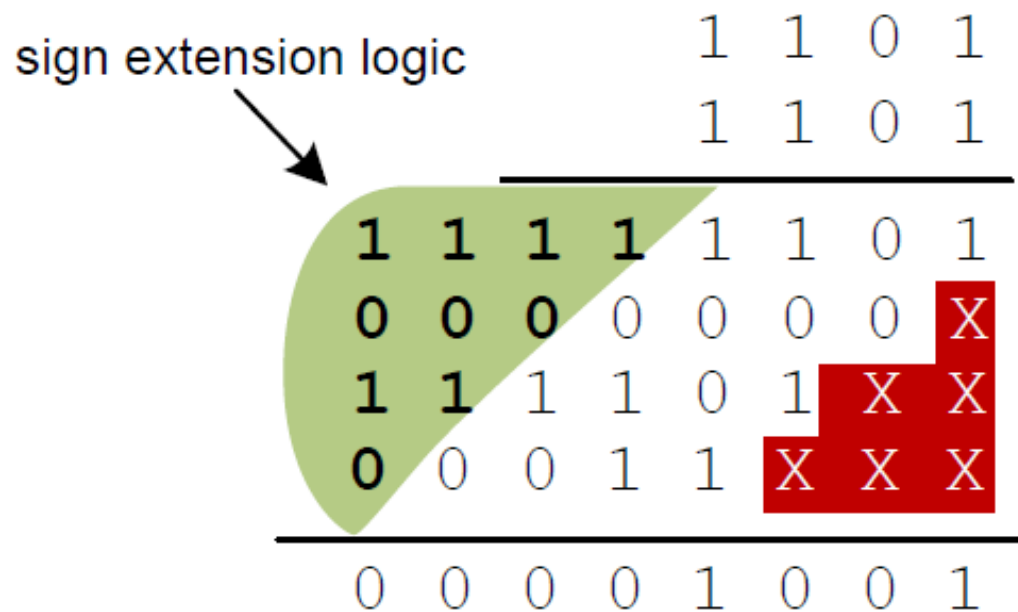


Stage 3



Two's Complement Signed Multiplier

- 4 x 4-bit signed by signed multiplication
 - The sign bits of the first three PPs are extended
 - Two's complement of the last PP is taken
 - HW implementation results in additional logic

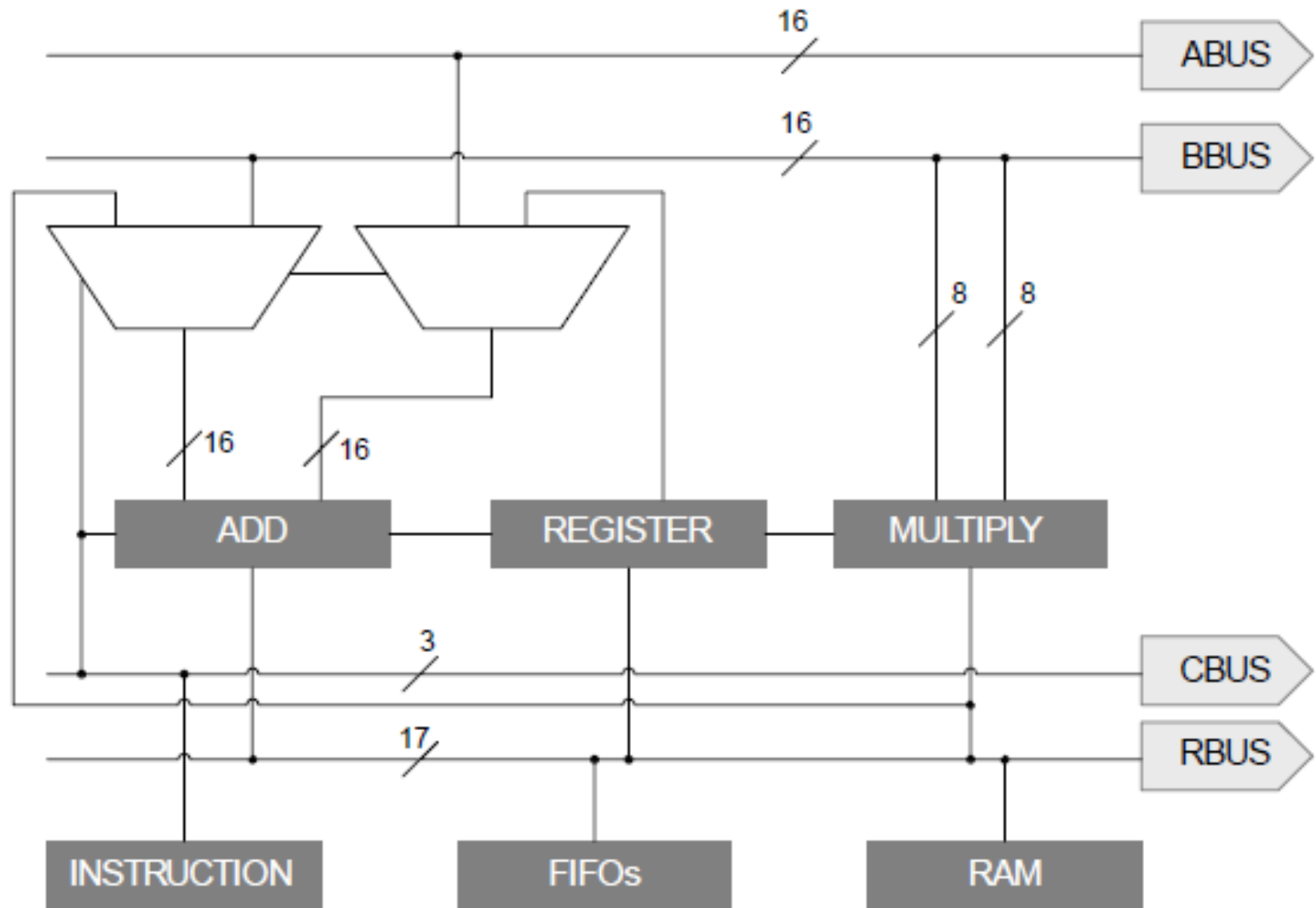


The End

Q&A

Appendix

8x8 multiplier and 16 bit adder



Tree diagram

Sign - extension Elimination

- Flip the sign bit, extend the number with all 1s and add a 1 at the location of the sign bit
- Irrespective of the sign of the number, the technique makes all the extended bits into 1s

extend all 1s flip the sign bit

$$\begin{array}{r}
 B = 000000 \\
 B = 111111 \\
 + \quad \quad \quad 1
 \end{array}
 \begin{array}{r}
 0.1101011 \\
 0.1101011 \\
 1
 \end{array}
 \begin{array}{r}
 0.1101011 \\
 0.1101011 \\
 1
 \end{array}$$

add 1 at the location of sign bit

$$000000.1101011$$

(a)

extend all 1s flip the sign bit

$$\begin{array}{r}
 B = 111111 \\
 B = 111111 \\
 + \quad \quad \quad 1
 \end{array}
 \begin{array}{r}
 1.1101011 \\
 1.1101011 \\
 1
 \end{array}
 \begin{array}{r}
 1.1101011 \\
 1.1101011 \\
 1
 \end{array}$$

add 1 at the location of sign bit

$$111111.1101011$$

(b)

Applied to multiplication

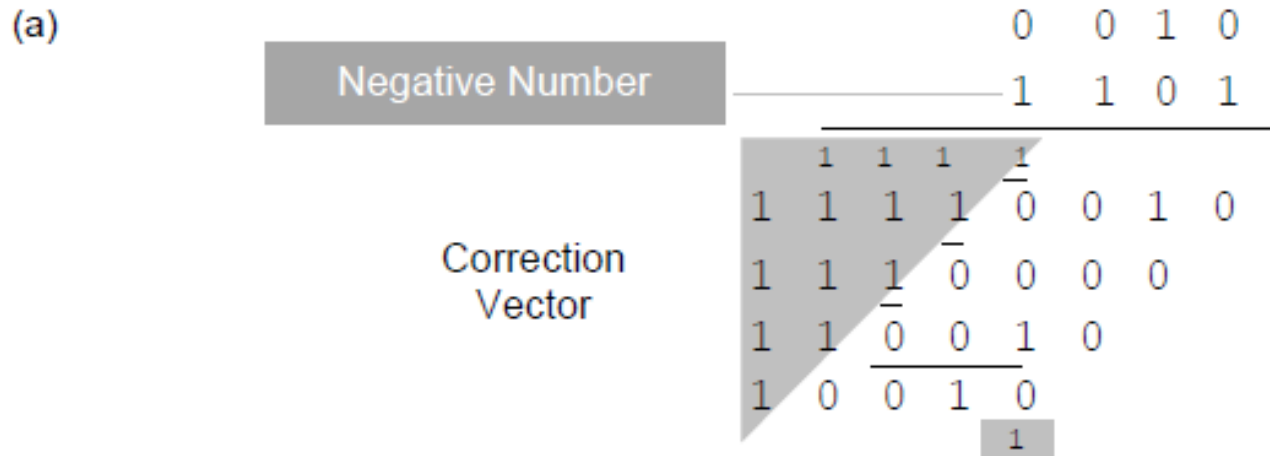
- First the MSB of all the PPs except the last one are flipped and a 1 is added at the sign bit location, and the number is extended by all 1s.
- For the last PP, the two's complement is computed by flipping all the bits and adding 1 to the LSB position.
- The MSB of the last PP is flipped again and 1 is added to this bit location for sign extension.
- All these 1s are added to find a correction vector (CV).
- Now all the 1s are removed and the CV is simply added and it takes care of the sign extension logic.

Correction vector

111111
11111SXXXXXXXXXXXXX
1111SXXXXXXXXXXXXX
111SXXXXXXXXXXXXX
11SXXXXXXXXXXXXX
1SXXXXXXXXXXXXX
SXXXXXXXXXXXXX
1
1's complement
and adding 1 at
LSB

$$\begin{array}{cccccccccccccccc}
 1 & 1 & 1 & 1 & 1 & 1 & & & & & & & & & & & & 1 \\
 1 & 1 & 1 & 1 & 1 & & & & & & & & & & & & & \\
 1 & 1 & 1 & 1 & & & & & & & & & & & & & & \\
 1 & 1 & 1 & & & & & & & & & & & & & & & \\
 1 & 1 & & & & & & & & & & & & & & & & \\
 1 & & & & & & & & & & & & & & & & & \\
 1 & & & & & & & & & & & & & & & & & \\
 \hline
 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0
 \end{array}$$

4x4 multiplication example



$$CV = 0001 \ 0000$$

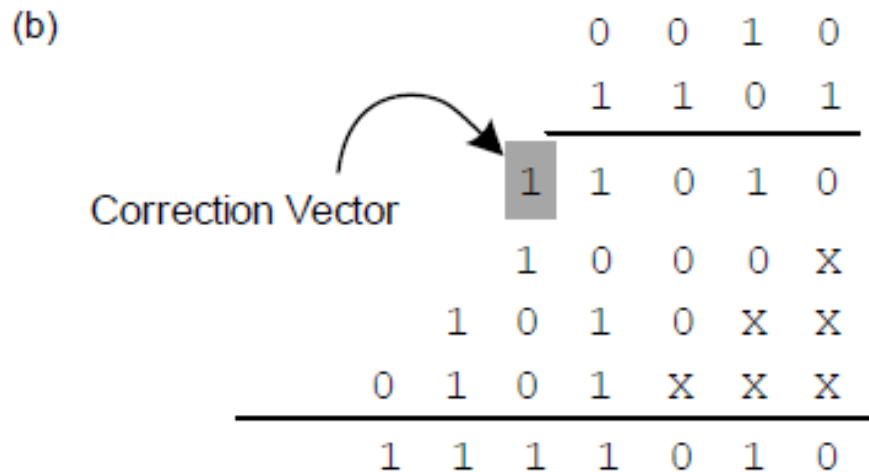


Figure 5.45 Multiplying two numbers, 0011 and 1101